Neural Network Solvers for Bayesian Estimation of New Keynesian DSGE Models:

A Self-Contained Treatment from Microfoundations to Hamiltonian Monte Carlo

With Line-by-Line Code Correspondence

Chak Wong

March 2026

Abstract

This document provides a complete, self-contained treatment of Bayesian estimation of Dynamic Stochastic General Equilibrium (DSGE) models using neural network solvers. Starting from microeconomic first principles, we derive the three-equation New Keynesian model from the optimisation problems of households, firms, and the central bank. We develop the perturbation solution method at both first and second order, explain the Kalman and Unscented Kalman filters for likelihood evaluation, and describe Hamiltonian Monte Carlo for posterior sampling. We then introduce a neural network that approximates the model's policy function conditioned on structural parameters, enabling rapid repeated evaluation during Bayesian inference.

The central technical challenge is ensuring that the neural network's gradients with respect to structural parameters are sufficiently smooth for gradient-based sampling. We identify two complementary ingredients for a production-quality neural DSGE solver. First, **canonical reparameterisation** ($\psi = M^{-1}(\theta - \theta_{\text{mode}})$, where $M = \Sigma^{1/2}$ is the posterior covariance square root) decorrelates parameters and equalises gradient scales, reducing HMC divergences 32-fold even without explicit gradient regularisation. Second, **perturbation gradient regularisation** directly trains the neural network's parameter derivatives to match the known-smooth perturbation gradients. Combined, these achieve zero HMC divergences with $\hat{R} = 1.018$ and ESS = 283.

A key new finding is that Sobolev regularisation alone—without any perturbation gradient target—passes all HMC diagnostics under canonical reparameterisation (0 divergences, $\hat{R} = 1.035$), broadening the applicability of neural DSGE solvers to models without perturbation solutions.

We systematically evaluate 30 training configurations spanning activation functions, architectures, regularisation strategies, multi-objective optimisation, parameter-space normalisation, and canonical reparameterisation. Several approaches from the physics-informed neural networks (PINNs) and multi-task learning literatures—Dual-Cone Gradient Descent, SIREN activations, spectral normalisation, gradient normalisation, and parameter-scale weighting—are analysed in detail, with precise explanations of why each fails in the DSGE context, confirmed under both the original θ-space and the reparameterised ψ-space.

The software includes an Epstein–Zin NK model extension that separates risk aversion from intertemporal substitution, demonstrating that the UKF is necessary when the excess risk aversion parameter $\delta = \gamma - 1/\psi$ is large.

All prior distributions, parameter transforms, hyperparameter settings, and experimental protocols are documented in full, enabling exact replication by third parties. Three appendices provide line-by-line documentation of the accompanying software, connecting every implementation detail to its mathematical equation and flagging non-obvious coding decisions.

Every equation in this document has been verified against the accompanying Python implementation, and code references are provided throughout. No prior knowledge of DSGE

models, Kalman filtering, Hamiltonian Monte Carlo, or neural network training is assumed. Every algorithm is derived from first principles.

Contents

I Economic and Mathematical Foundations **5**

1 Introduction **5**
1.1 What This Document Is About . . . 5
1.2 The Four-Stage Pipeline . . . 6
1.3 The Three Software Components . . . 6
1.4 Notation Conventions . . . 6
1.5 Parameter Vector and Code Mapping . . . 7

2 Mathematical Preliminaries **7**
2.1 Constrained Dynamic Optimisation and the Lagrangian . . . 7
2.2 The Euler Equation: Economic Intuition . . . 8
2.3 AR(1) Processes . . . 8
2.4 Log-Linearisation . . . 8
2.5 Kronecker Products . . . 9
2.6 Bayes' Theorem . . . 9
2.7 Why Markov Chain Monte Carlo? . . . 9
2.8 Neural Networks: A Brief Primer . . . 10
2.9 Latin Hypercube Sampling . . . 11
2.10 Multi-Objective Optimisation Background . . . 12
2.11 Eigendecomposition and Matrix Square Roots . . . 12

3 Microfoundations of the New Keynesian Model **13**
3.1 The Representative Household . . . 13
3.1.1 Deriving the Euler Equation . . . 13
3.2 Firms: Calvo Pricing and the Phillips Curve . . . 13
3.3 The Central Bank: Taylor Rule . . . 13
3.4 The Three-Equation System as Implemented . . . 14

4 Derivation of the Nonlinear IS Equation Used in the Code **14**
4.1 From Euler Equation to Nonlinear IS . . . 14
4.2 Recovering the Linear IS by Log-Linearisation . . . 14
4.3 The NKPC Residual . . . 14

5 The Epstein–Zin New Keynesian Model **14**

6 The SGU Small Open Economy Model **15**
6.1 Variables and Normalization . . . 15
6.2 Equilibrium Conditions . . . 16
6.3 Steady State . . . 17
6.4 Second-Order Perturbation: Why $\|H_{xx}\|$ and $\|G_{xx}\|$ Are Large . . . 18
6.4.1 Comparison with SGU (2004) . . . 18
6.4.2 Mechanism 1: The Hessian Tensor Ψ . . . 18
6.4.3 Mechanism 2: Near-Unit-Root Dynamics and Sylvester Amplification . . . 19
6.4.4 Mechanism 3: Mixed Log/Level Normalization . . . 19
6.4.5 Decomposition of H_{xx} and G_{xx} by Row . . . 19

6.4.6 Implications for Filtering and Estimation . . . 20
6.5 Joint Estimation: The HMCExamples.jl Approach . . . 20
6.5.1 Why Joint Estimation Avoids the H_{xx} Problem . . . 20
6.5.2 Dimension and Computational Cost . . . 21
6.5.3 Implications for This Codebase . . . 21

II Solution Methods **21**

7 The Policy Function Concept **21**

8 First-Order Perturbation Solution **21**
8.1 Method of Undetermined Coefficients . . . 21

9 Second-Order Perturbation Solution **22**

10 The Neural Network Solver **22**
10.1 Architecture . . . 22
10.2 Canonical Reparameterisation . . . 23
10.3 The All-in-One (AiO) Trick . . . 23
10.4 AiO Loss with Residual Balancing . . . 23
10.5 Training Pipeline . . . 24
10.6 Supervised Pretraining . . . 24
10.7 Coefficient Extraction Methods . . . 24

11 Gradient Regularisation **24**
11.1 The Gradient Quality Problem . . . 24
11.2 Sobolev Regularisation . . . 25
11.3 Perturbation Gradient Regularisation (PG) . . . 25
11.4 Gradient-Consistency Loss (GC) . . . 25

III Statistical Inference **25**

12 State-Space Models and Filtering **26**
12.1 The DSGE State-Space Form . . . 26
12.2 Pruned Second-Order State-Space . . . 26
12.3 The Kalman Filter . . . 26
12.4 The Square-Root Unscented Kalman Filter . . . 27
12.5 Pruned Square-Root UKF . . . 28
12.6 Measurement Noise Specification . . . 29

13 Bayesian Estimation **29**
13.1 Parameter Transforms and Prior Specification . . . 29
13.2 Hamiltonian Monte Carlo . . . 29
13.2.1 The Leapfrog Integrator . . . 30
13.2.2 Divergent Transitions . . . 30
13.2.3 Three-Phase HMC Pipeline (`MultiChainHMC`) . . . 30
13.3 MCMC Diagnostics . . . 30
13.4 Pass/Fail Criteria . . . 31

IV Experiments and Results **31**

14 Complete Experimental Protocol **31**
14.1 Data Generation . . . 31
14.2 Neural Network Training: Full Hyperparameters . . . 32
14.3 HMC Settings . . . 32

15 Experimental Results **33**
15.1 Results Summary . . . 33
15.2 Impact of Canonical Reparameterisation . . . 33
15.3 Multi-Seed Robustness . . . 33
15.4 GPU vs CPU Training . . . 34

16 Detailed Analysis of Failed Approaches **34**
16.1 Dual-Cone Gradient Descent (DCGD) . . . 34
16.2 Parameter-Scale-Weighted Gradient Penalties . . . 34
16.3 SIREN (Sinusoidal Representation Networks) . . . 35
16.4 MRePU (Multiply ReLU Power Units) . . . 35
16.5 Spectral Normalisation . . . 35
16.6 Perturbation-Corrected Architecture . . . 35
16.7 Combined GC + PG Regularisation . . . 35
16.8 AiO-Only Training Without Regularisation . . . 35
16.9 Summary: Taxonomy of Failure Modes . . . 36

17 Tutorial Results: Three Approaches Compared **36**

18 Lessons and Recommendations **36**
18.1 What Works . . . 36
18.2 What Does Not Work . . . 37
18.3 Corrected Beliefs from Reparameterisation Experiments . . . 37
18.4 Scaling to Large Models . . . 37

19 Complete Code–Equation Cross-Reference **38**

20 Conclusion **38**

A Detailed Documentation of `dsgeEZ.py` **39**
A.1 Backend and Utility Layer . . . 39
A.2 Model Classes . . . 39
A.2.1 `SmallNKModel` . . . 39
A.2.2 `EpsteinZinNKModel` . . . 39
A.3 Filter Classes . . . 40
A.4 Estimator Classes . . . 40
A.5 Diagnostic Functions . . . 40

B Detailed Documentation of `dsgeneural.py` **40**
B.1 Network Architectures . . . 41
B.1.1 `ScalablePolicyNetwork` . . . 41
B.1.2 `SIRENNetwork` . . . 41
B.2 Solver Classes . . . 41
B.2.1 `ConditionedNeuralNKSolver` . . . 41

C Detailed Documentation of `tutorial_neural_dsge.py` **42**
C.1 Overall Structure . . . 42
C.2 Case C: Neural Estimation (Three Stages) . . . 42

Part I
Economic and Mathematical Foundations

1 Introduction

1.1 What This Document Is About

Central banks around the world use mathematical models of the economy to forecast inflation, evaluate policy options, and communicate decisions. The dominant class of such models is called *Dynamic Stochastic General Equilibrium* (DSGE) models. The name describes three features: "dynamic" because agents make decisions over time, "stochastic" because random shocks hit the economy, and "general equilibrium" because all markets clear simultaneously.

Estimating the parameters of a DSGE model—for example, how aggressively the central bank raises interest rates in response to inflation—requires fitting the model to observed data. *Bayesian inference* is the standard method: it combines prior beliefs about parameter values with information from the data to produce a *posterior distribution*—a probability distribution over parameters given the observations.

This posterior distribution cannot be computed in closed form because the relationship between parameters and data involves solving a nonlinear expectational model (the DSGE model itself) and then filtering the observations through a state-space model. Instead, we draw samples from the posterior using *Markov chain Monte Carlo* (MCMC) methods, specifically *Hamiltonian Monte Carlo* (HMC). Each MCMC step requires:

1. proposing a new parameter value,
2. solving the DSGE model at that parameter value to obtain the "policy function" (how economic variables respond to shocks),
3. feeding the policy function into a filter (Kalman or Unscented Kalman) to compute how likely the observed data is under that parameterisation,
4. accepting or rejecting the proposal.

Since HMC may require thousands of such evaluations, the computational bottleneck is step 2: solving the model.

The innovation in this project is to replace step 2 with a *neural network* that has been trained, once and offline, to approximate the model's solution for any parameter value in a relevant region. After training, each evaluation in step 2 is a simple forward pass through the network—orders of magnitude faster than re-solving the model from scratch.

However, this introduces a subtle but critical challenge. HMC is a *gradient-based* sampler: it uses the gradient of the log-posterior with respect to parameters to guide its exploration. These gradients flow backward through the filter and then through the neural network's policy function. If the neural network's gradients with respect to the structural parameters are noisy or inaccurate—even if the function values themselves are accurate—the HMC sampler produces "divergent transitions" (numerical instabilities) and fails to explore the posterior correctly.

This document explains every component of this pipeline, from the economic theory underlying the DSGE model to the specific neural network training techniques that ensure gradient quality. Two key discoveries drive the design: first, *canonical reparameterisation* of the parameter space dramatically improves gradient quality by decorrelating parameters and equalising gradient scales; second, *perturbation gradient regularisation* provides a specific smooth target for the neural network's parameter derivatives.

1.2 The Four-Stage Pipeline

1. **Model** (Sections 3–4): Define the economic model—the IS curve, New Keynesian Phillips Curve, and Taylor rule—by deriving them from the optimisation problems of households, firms, and the central bank.

2. **Solve** (Sections 8–10): Find the *policy function* that maps the current economic state (productivity and monetary policy shocks) to outcomes (output, inflation, interest rates). Three methods: first-order perturbation (analytical), second-order perturbation (analytical with curvature), and neural network approximation (learned).

3. **Filter** (Section 12): Given a candidate policy function and observed data, compute the likelihood $p(Y|\boldsymbol{\theta})$—the probability of the observed data under parameter vector $\boldsymbol{\theta}$. The Kalman filter handles linear models; the Unscented Kalman Filter handles nonlinear ones.

4. **Estimate** (Section 13): Sample from the posterior $p(\boldsymbol{\theta}|Y) \propto p(Y|\boldsymbol{\theta})p(\boldsymbol{\theta})$ using HMC.

1.3 The Three Software Components

`dsgeEZ.py`**: Core framework.** Backend abstraction layer (`BackendRegistry`, `TensorFlowBackend`), model classes (`SmallNKModel`, `EpsteinZinNKModel`, `SGUEstimable`), four filters (`KalmanFilter`, `AugmentedKalmanFilter`, `SquareRootUKF`, `PrunedSquareRootUKF`), and three estimators (`MAPEstimator`, `HMCEstimator`, `MultiChainHMC`). Also provides MCMC diagnostics (`compute_rhat`, `compute_ess`, `mcmc_summary`). The backend abstraction enables future porting to JAX or PyTorch, while all current computation uses TensorFlow with `float64` precision.

`dsgeneural.py`**: Neural solver.** Two solver classes: `NeuralNKSolver` (fixed parameters, for testing) and `ConditionedNeuralNKSolver` (parameter-conditioned, for Bayesian estimation). Training pipeline includes supervised pretraining with cosine LR decay, early stopping, and curriculum learning; All-in-One residual minimisation; and multiple gradient regularisation strategies (Sobolev, perturbation gradient, gradient-consistency). Network architectures: `NeuralPolicyNetwork`, `ScalablePolicyNetwork` (dimension-adaptive with optional residual connections), `SIRENNetwork`. Three coefficient extraction methods: finite-difference stencil (`solve_fd`), autodiff Jacobian with FD Hessian (`solve_autodiff`), and ridge-regression Hessian (`solve_regression`, the default). Supports canonical reparameterisation (`reparam=True`, the default) for decorrelated ψ-space training.

`tutorial_neural_dsge.py`**: End-to-end tutorial.** Compares three estimation approaches: (A) linear model with Kalman filter, (B) 2nd-order perturbation with UKF, (C) neural solver with UKF. Case C uses a three-stage pipeline: quick perturbation posterior for theta-samples, train reparameterised neural solver, then full multi-chain HMC.

1.4 Notation Conventions

Scalars are denoted by lowercase italic (y, π, i). Vectors are lowercase bold ($\mathbf{x}$, $\boldsymbol{\theta}$). Matrices are uppercase italic (A, P). $\mathbb{E}_t[\cdot]$ denotes the conditional expectation given all information available at time t. The Kronecker product $x \otimes x$ produces a vector of all pairwise products (defined fully in Section 2.5). All variables denote *log-deviations from deterministic steady state* unless stated otherwise—that is, $y_t = 0$ means output is at its steady-state level. Throughout, $\boldsymbol{\theta}$ denotes structural parameters in constrained space, u denotes unconstrained parameters (for HMC), and ψ denotes canonically reparameterised parameters (for neural network training).

1.5 Parameter Vector and Code Mapping

The NK model has 9 structural parameters stored in the vector $\boldsymbol{\theta}$ with the following index convention, matching `SmallNKModel.NAMES` and the array ordering in all code:

Index	Code name	Math	Domain	Description	Code variable
0	`beta`	β	$(0.9, 1)$	Discount factor	`theta[0]`
1	`kappa`	κ	$(0, \infty)$	NKPC slope	`theta[1]`
2	`sigma_c`	σ_c	$(0, \infty)$	Inverse EIS (CRRA)	`theta[2]`
3	`phi_pi`	ϕ_π	$(1, \infty)$	Taylor rule: inflation	`theta[3]`
4	`phi_y`	ϕ_y	$(0, \infty)$	Taylor rule: output	`theta[4]`
5	`rho_a`	ρ_a	$(-1, 1)$	Productivity persistence	`theta[5]`
6	`rho_v`	ρ_v	$(-1, 1)$	Policy shock persistence	`theta[6]`
7	`sig_a`	σ_a	$(0, \infty)$	Productivity shock std	`theta[7]`
8	`sig_v`	σ_v	$(0, \infty)$	Policy shock std	`theta[8]`

The default test values used in `tutorial_neural_dsge.py` (`THETA_TRUE`) and `dsgeneural.py` (`main`) are:

$$\boldsymbol{\theta}_{\text{true}} = (0.99, 0.30, 1.00, 1.50, 0.125, 0.90, 0.50, 0.05, 0.05). \tag{1}$$

The tutorial uses $\sigma_c = 1.0$ and $\sigma_a = \sigma_v = 0.05$; the `dsgeneural.py` test harness uses $\sigma_c = 2.0$ and $\sigma_a = \sigma_v = 0.10$.

The software also includes an `EpsteinZinNKModel` with 10 parameters (adding γ and ψ for recursive preferences, replacing σ_c); see Section 5.

2 Mathematical Preliminaries

This section collects the mathematical tools used throughout the document. Readers familiar with these topics may skim or skip ahead.

2.1 Constrained Dynamic Optimisation and the Lagrangian

Much of macroeconomics involves agents choosing sequences of actions over time to maximise an objective subject to constraints. The standard tool is the *Lagrangian method*.

Suppose a household wants to maximise $f(C, N)$ subject to a constraint $g(C, N) \leq 0$. The Lagrangian is:

$$\mathcal{L} = f(C, N) - \lambda\, g(C, N), \tag{2}$$

where $\lambda \geq 0$ is the *Lagrange multiplier* (also called the "shadow price"—it measures how much the objective improves if the constraint is relaxed by one unit). At the optimum, $\partial\mathcal{L}/\partial C = 0$ and $\partial\mathcal{L}/\partial N = 0$.

In the dynamic setting, the household faces a constraint at *every* time period $t = 0, 1, 2, \ldots$, so we have a separate multiplier λ_t for each period. The infinite-horizon Lagrangian is:

$$\mathcal{L} = \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t \big[U(C_t, N_t) - \lambda_t \cdot g_t(C_t, N_t, B_t, \ldots)\big]. \tag{3}$$

Taking the first-order condition (FOC) with respect to C_t and B_t (the bond holding), and then eliminating λ_t between consecutive periods, yields the *Euler equation*—the fundamental optimality condition relating today's marginal utility to tomorrow's expected marginal utility.

2.2 The Euler Equation: Economic Intuition

The Euler equation captures a simple but powerful idea: *at the optimum, the household is indifferent between consuming a little more today and saving that amount for tomorrow.*

If the household consumes one more unit today, the gain in utility is the *marginal utility of consumption today*, $U'(C_t)$. If instead the household saves that unit (lending it at gross interest rate $1 + i_t$, adjusting for inflation $1 + \pi_{t+1}$), the gain is the *expected discounted marginal utility tomorrow*:

$$\beta(1 + i_t)\,\mathbb{E}_t\left[\frac{U'(C_{t+1})}{1 + \pi_{t+1}}\right]. \tag{4}$$

At the optimum, these must be equal:

$$U'(C_t) = \beta(1 + i_t)\,\mathbb{E}_t\left[\frac{U'(C_{t+1})}{1 + \pi_{t+1}}\right]. \tag{5}$$

If the right-hand side were larger, the household would want to save more (consuming less today, more tomorrow); if smaller, it would want to save less. Only at equality is there no incentive to adjust.

2.3 AR(1) Processes

Many economic shocks are modelled as *autoregressive processes of order 1* (AR(1)). The idea is that today's shock depends on yesterday's shock plus a random innovation:

$$x_t = \rho\, x_{t-1} + \sigma\, \varepsilon_t, \qquad \varepsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1). \tag{6}$$

The parameter ρ controls *persistence*: when ρ is close to 1, shocks die out slowly; when ρ is close to 0, each period's value is almost entirely determined by the current innovation. The condition $|\rho| < 1$ ensures *stationarity*—the process does not explode or drift to infinity.

Key properties used throughout:

$$\mathbb{E}_t[x_{t+1}] = \rho\, x_t, \tag{7}$$

$$\mathrm{Var}(x_t) = \frac{\sigma^2}{1 - \rho^2} \quad \text{(unconditional variance)}. \tag{8}$$

Property (7) follows from $\mathbb{E}_t[\varepsilon_{t+1}] = 0$. Property (8) comes from iterating $\mathrm{Var}(x_t) = \rho^2 \mathrm{Var}(x_{t-1}) + \sigma^2$ to its fixed point.

Equation (8) is used in `_compute_domain_bounds` and `NeuralNKSolver.train` to set the sampling domain for neural network training: the state grid covers $\pm 3.5 \times \sigma/\sqrt{1 - \rho^2}$, capturing 99.95% of the stationary distribution.

2.4 Log-Linearisation

DSGE models involve nonlinear equations (e.g., $C_t^{-\sigma_c}$, products like $C_t \cdot P_t$). *Log-linearisation* approximates these around the steady state using a first-order Taylor expansion in logs.

Define the *log-deviation* $\hat{x}_t = \log(X_t/\bar{X})$, where $\bar{X}$ is the steady-state value. Then $X_t = \bar{X}\exp(\hat{x}_t) \approx \bar{X}(1 + \hat{x}_t)$ for small deviations. Useful identities:

$$X_t^{-\sigma_c} \approx \bar{X}^{-\sigma_c}(1 - \sigma_c \hat{x}_t), \tag{9}$$

$$X_t \cdot Y_t \approx \bar{X}\bar{Y}(1 + \hat{x}_t + \hat{y}_t), \tag{10}$$

$$X_t/Y_t \approx (\bar{X}/\bar{Y})(1 + \hat{x}_t - \hat{y}_t). \tag{11}$$

After log-linearisation, the model becomes a system of linear expectational difference equations, which can be solved analytically (Section 8). Throughout this document, all variables without overbars (y_t, π_t, etc.) are log-deviations from steady state.

2.5 Kronecker Products

The *Kronecker product* (or *tensor product*) of two vectors produces a vector containing all pairwise products of their elements. For $x = (a, v)^\top$:

$$x \otimes x = \begin{pmatrix} a \\ v \end{pmatrix} \otimes \begin{pmatrix} a \\ v \end{pmatrix} = \begin{pmatrix} a^2 \\ av \\ va \\ v^2 \end{pmatrix} \in \mathbb{R}^4. \tag{12}$$

The reason we use Kronecker products is that a *second-order* Taylor expansion of a function $f(x)$ around the origin takes the form:

$$f(x) \approx f(0) + \underbrace{J \cdot x}_{\text{linear}} + \frac{1}{2} \underbrace{H \cdot (x \otimes x)}_{\text{quadratic}}, \tag{13}$$

where J is the Jacobian (matrix of first derivatives) and H is the Hessian reshaped into a matrix that multiplies the Kronecker product. This is just the multivariate Taylor expansion written in a compact matrix form.

In code, the Kronecker product is computed as `np.outer(x, x).ravel()` (in `simulate`) or `x[:, :, None] * x[:, None, :]` reshaped to `[-1, ns**2]` (in batched training loops). The 4 columns of G_{xx} and H_{xx} correspond to $(a \otimes a,\ a \otimes v,\ v \otimes a,\ v \otimes v)$ in this order.

2.6 Bayes' Theorem

Bayes' theorem is the foundation of Bayesian inference. Given observed data Y and parameters $\boldsymbol{\theta}$:

$$\underbrace{p(\boldsymbol{\theta}|Y)}_{\text{posterior}} = \frac{\overbrace{p(Y|\boldsymbol{\theta})}^{\text{likelihood}} \times \overbrace{p(\boldsymbol{\theta})}^{\text{prior}}}{\underbrace{p(Y)}_{\text{evidence}}}. \tag{14}$$

The *prior* $p(\boldsymbol{\theta})$ encodes our beliefs about the parameters before seeing the data (e.g., "the discount factor is probably close to 0.99"). The *likelihood* $p(Y|\boldsymbol{\theta})$ measures how probable the observed data is under parameter value $\boldsymbol{\theta}$. The *posterior* $p(\boldsymbol{\theta}|Y)$ is the updated belief after seeing the data. The *evidence* $p(Y) = \int p(Y|\boldsymbol{\theta})p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$ is a normalising constant that does not depend on $\boldsymbol{\theta}$ and is extremely difficult to compute in high dimensions.

Since the evidence is a constant, we write:

$$p(\boldsymbol{\theta}|Y) \propto p(Y|\boldsymbol{\theta}) \times p(\boldsymbol{\theta}). \tag{15}$$

This proportionality is sufficient for MCMC sampling, which only requires the ratio of posterior densities at different parameter values (the normalising constant cancels).

In our pipeline, the likelihood $p(Y|\boldsymbol{\theta})$ is computed by the Kalman or Unscented Kalman filter (Section 12), and the prior is specified in unconstrained space (Section 13.1).

2.7 Why Markov Chain Monte Carlo?

The posterior $p(\boldsymbol{\theta}|Y)$ is a 9-dimensional distribution with no closed-form expression. We cannot compute posterior means, credible intervals, or other summaries analytically. Instead, we generate a sequence of *samples* $\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \ldots, \boldsymbol{\theta}^{(N)}$ from this distribution and approximate:

$$\mathbb{E}[\boldsymbol{\theta}|Y] \approx \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{\theta}^{(i)}, \qquad \mathrm{Var}[\boldsymbol{\theta}|Y] \approx \frac{1}{N-1} \sum_{i=1}^{N} (\boldsymbol{\theta}^{(i)} - \bar{\boldsymbol{\theta}})^2. \tag{16}$$

Markov chain Monte Carlo (MCMC) generates these samples by constructing a Markov chain whose stationary distribution is the target posterior. The simplest version, the *Metropolis–Hastings algorithm*, works as follows:

1. Start at some $\boldsymbol{\theta}^{(0)}$.
2. **Propose**: draw $\boldsymbol{\theta}^* \sim q(\cdot|\boldsymbol{\theta}^{(i)})$ from a proposal distribution (e.g., a Gaussian centred at the current value).
3. **Accept/Reject**: compute the acceptance probability

$$\alpha = \min\left(1, \frac{p(\boldsymbol{\theta}^*|Y)\, q(\boldsymbol{\theta}^{(i)}|\boldsymbol{\theta}^*)}{p(\boldsymbol{\theta}^{(i)}|Y)\, q(\boldsymbol{\theta}^*|\boldsymbol{\theta}^{(i)})}\right) \tag{17}$$

   and set $\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^*$ with probability α, otherwise $\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)}$.

The problem with simple Metropolis–Hastings using a random-walk proposal ($q(\boldsymbol{\theta}^*|\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}, \epsilon^2 I)$) is that it explores the parameter space very slowly. Small step sizes ϵ give high acceptance rates but move slowly; large step sizes move fast but proposals are usually rejected because they land in low-probability regions. In 9 dimensions, this trade-off becomes severe.

Hamiltonian Monte Carlo (Section 13.2) solves this problem by using gradient information to make large, directed proposals that are almost always accepted.

2.8 Neural Networks: A Brief Primer

A *feedforward neural network* is a parameterised function $f_{\boldsymbol{\psi}} : \mathbb{R}^d \to \mathbb{R}^m$ composed of alternating linear transformations and nonlinear activations. For a network with L hidden layers:

$$h_0 = x \quad \text{(input)}, \tag{18}$$

$$h_\ell = \phi(W_\ell\, h_{\ell-1} + b_\ell), \quad \ell = 1, \ldots, L, \tag{19}$$

$$f_{\boldsymbol{\psi}}(x) = W_{L+1}\, h_L + b_{L+1} \quad \text{(output, no activation)}, \tag{20}$$

where W_ℓ are weight matrices, b_ℓ are bias vectors, and ϕ is the activation function. The collection of all weights and biases is $\boldsymbol{\psi} = \{W_1, b_1, \ldots, W_{L+1}, b_{L+1}\}$.

Common activation functions include:

$$\mathrm{ReLU}(z) = \max(0, z) \qquad \text{(not differentiable at 0)}, \tag{21}$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \qquad \text{(smooth, } C^\infty\text{)}, \tag{22}$$

$$\mathrm{sigmoid}(z) = \frac{1}{1 + e^{-z}} \qquad \text{(smooth, output in (0,1))}. \tag{23}$$

The *universal approximation theorem* states that a sufficiently wide single-hidden-layer network can approximate any continuous function on a compact set to arbitrary accuracy. In practice, deeper networks with moderate width are more efficient.

Training. The network parameters $\boldsymbol{\psi}$ are found by minimising a *loss function* $\mathcal{L}(\boldsymbol{\psi})$ using gradient descent:

$$\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} - \alpha \nabla_{\boldsymbol{\psi}} \mathcal{L}(\boldsymbol{\psi}), \tag{24}$$

where α is the *learning rate*. The gradient $\nabla_{\boldsymbol{\psi}} \mathcal{L}$ is computed by the *backpropagation algorithm*, which is just the chain rule applied systematically through the network layers. Modern implementations use *automatic differentiation* (autodiff), which computes exact derivatives (not finite differences) at machine precision.

Adam optimiser. Adam (Kingma and Ba, 2015) is a variant of gradient descent that maintains running averages of the gradient (first moment m) and its square (second moment v), and uses these to adapt the step size for each parameter:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{25}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{26}$$

$$\boldsymbol{\psi}_t = \boldsymbol{\psi}_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \tag{27}$$

where $g_t = \nabla_{\boldsymbol{\psi}} \mathcal{L}$, $\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$ are bias-corrected estimates, and $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ are standard defaults. Parameters with large gradients get smaller effective step sizes, and vice versa, which helps when different parameters have different scales.

Cosine learning rate decay. Instead of using a fixed learning rate α, the learning rate is reduced during training following a cosine schedule:

$$\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right), \tag{28}$$

where t is the current epoch, T is the total number of epochs, $\alpha_{\max}$ is the initial learning rate, and $\alpha_{\min}$ is the final learning rate. This starts at $\alpha_{\max}$, smoothly decays to $\alpha_{\min}$ (typically 0 or 10^{-6}), and helps the optimiser settle into a good minimum.

Weight decay. Adding a penalty $\lambda_{\text{wd}} \|\boldsymbol{\psi}\|^2$ to the loss function discourages large weights, which reduces overfitting and improves the smoothness of the learned function. Equivalently, the gradient update becomes $\boldsymbol{\psi} \leftarrow (1 - \alpha\lambda_{\text{wd}})\boldsymbol{\psi} - \alpha \nabla_{\boldsymbol{\psi}} \mathcal{L}$.

Gradient clipping. If the gradient norm $\|\nabla_{\boldsymbol{\psi}} \mathcal{L}\|$ exceeds a threshold $G_{\max}$, the gradient is rescaled: $g \leftarrow g \cdot G_{\max}/\|g\|$. This prevents catastrophically large updates that can destabilise training. In our code, $G_{\max} = 10$.

2.9 Latin Hypercube Sampling

When sampling points to train the neural network, we want good coverage of the input space. Simple random sampling can leave large gaps, especially in high dimensions. *Latin Hypercube Sampling* (LHS) provides much more uniform coverage.

The idea is simple. Suppose we want N points in $[0,1]^d$. For each dimension j, divide $[0,1]$ into N equal bins. Then randomly assign each of the N points to a different bin in each dimension, and place the point uniformly within its bin.

Formally: for point i in dimension j:

$$x_{ij} = \frac{\pi_j(i) - 1 + U_{ij}}{N}, \qquad U_{ij} \sim \text{Uniform}(0,1), \tag{29}$$

where π_j is a random permutation of $\{1, \ldots, N\}$, independent for each dimension j.

The result is that the projection of the sample onto any single dimension is exactly uniform (one point per bin), while the joint distribution provides good d-dimensional coverage. LHS is used in three places in the code: `_lhs_state_grid` generates state-space training points; `_lhs_gaussian` generates Gaussian shock vectors ε by transforming LHS uniform samples through the inverse normal CDF $\varepsilon = \Phi^{-1}(x_{\text{LHS}})$; and parameter sampling in `_sample_theta` uses LHS mapped through the inverse normal CDF to the inflated posterior Gaussian.

2.10 Multi-Objective Optimisation Background

Training the neural solver involves minimising a composite loss $\mathcal{L} = \sum_k \lambda_k \mathcal{L}_k$ comprising several terms: the AiO Euler residual, Sobolev regularisation, perturbation gradient matching, and weight decay. Each term has its own gradient with respect to the network weights:

$$g_k = \nabla_\psi \mathcal{L}_k, \qquad k = 1, \ldots, K. \tag{30}$$

The standard approach simply sums these: $g = \sum_k \lambda_k g_k$. Several alternative strategies from the multi-task learning and physics-informed neural networks (PINNs) literatures have been proposed when the loss terms "conflict"—that is, when reducing one loss increases another. We will describe and analyse these alternatives in Section 16.

Two key concepts from convex geometry are needed to understand these alternatives.

Cones and dual cones. The *conic hull* of a set of vectors $\{v_1, \ldots, v_K\}$ is the set of all non-negative linear combinations:

$$\mathrm{cone}(v_1, \ldots, v_K) = \Big\{ \sum_{k=1}^{K} \alpha_k v_k \;\Big|\; \alpha_k \geq 0 \Big\}. \tag{31}$$

The *dual cone* of a set S is the set of all vectors that form non-negative inner products with every vector in S:

$$S^* = \{w \in \mathbb{R}^d : w^\top v \geq 0 \;\; \forall\, v \in S\}. \tag{32}$$

Geometrically, S^* is the set of directions that "do not oppose" any direction in S. If S consists of a single vector v, then S^* is the half-space $\{w : w^\top v \geq 0\}$ (all directions within 90 degrees of v). If $S = \{v_1, v_2\}$ and $v_1^\top v_2 > 0$ (the vectors roughly agree), then S^* is a wide cone; if $v_1^\top v_2 < 0$ (the vectors oppose each other), S^* is narrow or empty.

Projection onto the dual cone. Given a vector g and a set S, the *projection* of g onto S^* is the closest point in S^* to g:

$$\mathrm{proj}_{S^*}(g) = \arg\min_{w \in S^*} \|w - g\|^2. \tag{33}$$

If g is already in S^* (i.e., $g^\top v \geq 0$ for all $v \in S$), then the projection is g itself—no modification is needed. If $g \notin S^*$ (i.e., g opposes at least one vector in S), the projection removes the opposing component. Computing this projection requires solving a quadratic programme, which for two vectors reduces to a simple formula (Section 16.1).

2.11 Eigendecomposition and Matrix Square Roots

The canonical reparameterisation (Section 10.2) relies on the eigendecomposition of the posterior covariance matrix. Given a symmetric positive-definite matrix $\Sigma \in \mathbb{R}^{d \times d}$, its eigendecomposition is:

$$\Sigma = V \Lambda V^\top, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d), \qquad V^\top V = I, \tag{34}$$

where $\lambda_j > 0$ are the eigenvalues and V contains the corresponding orthonormal eigenvectors.

The *matrix square root* is:

$$M = \Sigma^{1/2} = V \Lambda^{1/2} V^\top, \qquad \Lambda^{1/2} = \mathrm{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_d}), \tag{35}$$

satisfying $MM^\top = \Sigma$. The inverse square root is $M^{-1} = V \Lambda^{-1/2} V^\top$.

In the code, the eigendecomposition is computed using `np.linalg.eigh`, and the matrix M used for reparameterisation is stored as $M = V\Lambda^{1/2}$ (without the trailing $V^\top$), giving $MM^\top = V\Lambda V^\top = \Sigma$ as required.

3 Microfoundations of the New Keynesian Model

The New Keynesian model is built up from three blocks: a representative household that decides how much to consume and work, a continuum of firms that set prices, and a central bank that sets interest rates. We derive each block from first principles.

3.1 The Representative Household

We assume a single "representative household" that stands in for the average behaviour of all households in the economy. This household makes two decisions each period: how much to consume (C_t) and how much to work (N_t). It derives utility from consumption and disutility from labour.

The household maximises the expected sum of discounted lifetime utility:

$$\mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t \left[\frac{C_t^{1-\sigma_c}}{1-\sigma_c} - \chi \frac{N_t^{1+\varphi}}{1+\varphi} \right], \tag{36}$$

where C_t is consumption, N_t is hours worked, $\beta \in (0,1)$ is the discount factor, $\sigma_c > 0$ is the coefficient of relative risk aversion (CRRA), $\varphi \geq 0$ is the inverse Frisch elasticity, and $\chi > 0$ is a scaling parameter for the disutility of labour (drops out in the log-linearised model).

The household faces the *budget constraint* each period:

$$P_t C_t + \frac{B_t}{1+i_t} = B_{t-1} + W_t N_t + D_t, \tag{37}$$

where P_t is the aggregate price level, B_t is the nominal value of one-period bonds, i_t is the nominal interest rate, W_t is the nominal wage, and D_t is dividend income.

3.1.1 Deriving the Euler Equation

Forming the Lagrangian, taking first-order conditions for C_t and B_t, and combining yields:

$$1 = \beta(1+i_t)\,\mathbb{E}_t \left[\left(\frac{C_{t+1}}{C_t} \right)^{-\sigma_c} \frac{1}{1+\pi_{t+1}} \right]. \tag{38}$$

3.2 Firms: Calvo Pricing and the Phillips Curve

In the New Keynesian model, there is a continuum of firms indexed by $j \in [0,1]$, each producing a slightly different variety of the consumption good. Each period, each firm independently: with probability $1-\xi$ resets its price optimally, and with probability ξ keeps its previous price.

Log-linearising the optimal reset price equation and the aggregate price level identity yields the New Keynesian Phillips Curve:

$$\pi_t = \beta\,\mathbb{E}_t[\pi_{t+1}] + \kappa\, y_t, \qquad \kappa = \frac{(1-\xi)(1-\beta\xi)}{\xi}(\sigma_c + \varphi). \tag{39}$$

In the code, κ is treated as a free parameter.

3.3 The Central Bank: Taylor Rule

$$i_t = \phi_\pi\, \pi_t + \phi_y\, y_t + v_t. \tag{40}$$

The constraint $\phi_\pi > 1$ (Taylor principle) ensures determinacy. The Taylor rule is enforced analytically in both the perturbation solver and the neural solver.

3.4 The Three-Equation System as Implemented

State dynamics:

$$x_t = h_x\, x_{t-1} + \eta\, \varepsilon_t, \quad h_x = \begin{pmatrix} \rho_a & 0 \\ 0 & \rho_v \end{pmatrix}, \quad \eta = \begin{pmatrix} \sigma_a & 0 \\ 0 & \sigma_v \end{pmatrix}. \tag{41}$$

IS curve:

$$y_t = \mathbb{E}_t[y_{t+1}] - \frac{1}{\sigma_c}(i_t - \mathbb{E}_t[\pi_{t+1}]) + (1-\rho_a)\, a_t. \tag{42}$$

NKPC:

$$\pi_t = \kappa\, y_t + \beta\, \mathbb{E}_t[\pi_{t+1}]. \tag{43}$$

Taylor rule:

$$i_t = \phi_\pi\, \pi_t + \phi_y\, y_t + v_t. \tag{44}$$

4 Derivation of the Nonlinear IS Equation Used in the Code

4.1 From Euler Equation to Nonlinear IS

$$\boxed{\underbrace{\exp(-\sigma_c y_t - i_t + \sigma_c(1-\rho_a)a_t)}_{\text{LHS: deterministic}} = \mathbb{E}_t\left[\underbrace{\exp(-\sigma_c y_{t+1} + \sigma_c\sigma_a\varepsilon_{a,t+1} - \pi_{t+1})}_{\text{RHS: stochastic}}\right].} \tag{45}$$

4.2 Recovering the Linear IS by Log-Linearisation

Taking logs and applying certainty equivalence recovers (42).

4.3 The NKPC Residual

$$R_{\text{NKPC}} = \pi_t - \kappa y_t - \beta\pi_{t+1}. \tag{46}$$

5 The Epstein–Zin New Keynesian Model

The software also includes an `EpsteinZinNKModel` that uses recursive Epstein–Zin preferences to separate risk aversion (γ) from the elasticity of intertemporal substitution (ψ). Under standard CRRA, these are linked as $\gamma = 1/\psi = \sigma_c$.

The key parameter governing all second-order effects is the *excess risk aversion*:

$$\delta = \gamma - 1/\psi. \tag{47}$$

When $\delta = 0$, the model reduces to standard CRRA and all second-order corrections vanish. When $\delta > 0$ (preference for early resolution of uncertainty), the second-order corrections in H_{xx}, G_{xx}, g_{ss}, and h_{ss} are all proportional to δ. The Epstein–Zin model also introduces cross-state interaction terms ($a \times v$ in H_{xx}, G_{xx}) that are absent under CRRA, because the nonlinear stochastic discount factor prices the interaction of real and nominal shocks.

This model demonstrates why the UKF is necessary: when $\gamma = 10$ and $\psi = 1.5$, the excess risk aversion $\delta \approx 9.33$ amplifies all second-order corrections dramatically, and the linear Kalman filter produces severely biased posteriors. The `dsgeEZ.py` main function runs a comprehensive comparison of KF, Augmented KF, and SR-UKF for the Epstein–Zin model.

6 The SGU Small Open Economy Model

The `SGUEstimable` class implements the small open economy model from Schmitt-Grohé and Uribe Schmitt-Grohé and Uribe (2003), extended with a consumption preference shock (μ) and a risk premium shock (ζ) following Childers et al. Childers et al. (2024). This is a substantially larger and more nonlinear model than the 3-equation New Keynesian model: it has 7 state variables, 8 control variables, 3 shocks, and 15 equilibrium conditions.

6.1 Variables and Normalization

State variables ($n_x = 7$).

Index	Symbol	Normalization	Description
0	d_t	Level	Foreign debt
1	k_t	Log	Capital stock (e^{k_t} is the level)
2	r_t	Log	Net interest rate (e^{r_t} is the level)
3	a_t	Log	Total factor productivity (AR(1))
4	rp_t	Level	Risk premium
5	ζ_t	Level	Risk premium shock (AR(1))
6	μ_t	Level	Preference shock (AR(1))

Control variables ($n_y = 8$).

Index	Symbol	Normalization	Description
0	c_t	Log	Consumption (e^{c_t} is the level)
1	h_t	Log	Hours (e^{h_t} is the level)
2	GDP_t	Log	Output (e^{GDP_t} is the level)
3	i_t	Log	Investment (e^{i_t} is the level)
4	k_t^f	Log	Future capital (timing convention)
5	λ_t	Log	Marginal utility (e^{λ_t} is the level)
6	tb_t	Level	Trade balance / GDP
7	ca_t	Level	Current account / GDP

Mixed normalization. A crucial feature of this model is its *mixed log/level normalization*: variables d, rp, ζ, μ, tb, ca are in levels, while k, r, a, c, h, GDP, i, k^f, λ are in logs. This means the equilibrium equations contain $e^{(\cdot)}$ terms that introduce strong nonlinearity when the Hessian is evaluated. In contrast, fully log-linearized models (such as the standard RBC and the 3-equation NK model) have $\Psi \approx 0$ in many equations, producing small or zero H_{xx} and G_{xx}.

Structural parameters (15 total).

$\alpha = 0.32$	Capital share	$\delta = 0.1$	Depreciation rate
$\gamma = 2$	Risk aversion	$\phi = 0.028$	Capital adj. cost
$\omega = 1.455$	Labour supply elast.	$r_w = 0.04$	World interest rate
$\psi = 7.42 \times 10^{-4}$	Debt elasticity	$\bar{d} = 0.7442$	Steady-state debt
$\beta = 1/(1 + r_w)$	Discount factor		
$\rho = 0.42$	TFP persistence	$\sigma_e = 0.0129$	TFP shock std
$\rho_u = 0.2$	Risk prem. persist.	$\sigma_u = 0.003$	Risk prem. shock std
$\rho_v = 0.4$	Pref. shock persist.	$\sigma_v = 0.1$	Pref. shock std

6.2 Equilibrium Conditions

The model is defined by 15 equilibrium conditions $H(y', y, x', x; \theta) = 0$. Following the ordering in `SGUSmallOpen.residuals`, with level values written as $C = e^c$, $K = e^k$, etc.:

Eq. 1: Resource constraint.

$$-C - I - \tfrac{\phi}{2}(K^f - K)^2 - (1 + e^r)\, d + d' + Y = 0. \tag{48}$$

Output minus consumption, investment, capital adjustment costs, and debt service equals the change in debt.

Eq. 2: Production function.

$$Y - K^\alpha H^{1-\alpha} e^a = 0. \tag{49}$$

Cobb–Douglas technology with TFP shock a_t.

Eq. 3: Capital accumulation.

$$K^f - I - (1 - \delta)K = 0. \tag{50}$$

Eq. 4: Euler equation (intertemporal).

$$\Lambda - \beta(1 + e^{r'})e^\mu \Lambda' = 0. \tag{51}$$

The preference shock e^μ enters multiplicatively, amplifying intertemporal substitution.

Eq. 5: Marginal utility (GHH preferences).

$$(C - H^\omega/\omega)^{-\gamma} - \Lambda = 0. \tag{52}$$

GHH (Greenwood–Hercowitz–Huffman) preferences eliminate the wealth effect on labour supply. The composite $C - H^\omega/\omega$ must be positive.

Eq. 6: Labour FOC (intratemporal).

$$H^{\omega-1}(C - H^\omega/\omega)^{-\gamma} - (1 - \alpha)\, Y\, \Lambda/H = 0. \tag{53}$$

From GHH preferences, the marginal rate of substitution between consumption and leisure equals the real wage.

Eq. 7: Capital FOC (Tobin's q).

$$(1 + \phi(K^f - K))\Lambda - \beta\big((1 - \delta) + \phi(K^{f\prime} - K^f) + \alpha Y'/K^f\big)e^\mu \Lambda' = 0. \tag{54}$$

Capital adjustment costs create a wedge between the marginal cost of investment (left) and the marginal return (right).

Eq. 8: Interest rate.

$$e^{r'} - r_w - \mathrm{rp}' = 0. \tag{55}$$

The country-specific interest rate equals the world rate plus a risk premium.

Eq. 9: Risk premium.

$$\mathrm{rp}' - \psi(e^{d'-\bar{d}} - 1) - \zeta = 0. \tag{56}$$

The risk premium is increasing in debt (the "elastic interest rate" closure of SGU Schmitt-Grohé and Uribe (2003)) plus a stochastic component ζ.

Eq. 10: Trade balance.

$$\mathrm{tb} + \frac{C + I + \frac{\phi}{2}(K^f - K)^2}{Y} - 1 = 0. \tag{57}$$

Eq. 11: Current account.

$$\mathrm{ca} + \frac{d' - d}{Y} = 0. \tag{58}$$

Eq. 12: Capital identity.

$$K^f - K' = 0. \tag{59}$$

Links the control k_t^f (chosen at time t) to the state k_{t+1}.

Eqs. 13–15: Exogenous shocks.

$$a_{t+1} = \rho\, a_t + \sigma_e\, \varepsilon_{t+1}^a, \tag{60}$$

$$\zeta_{t+1} = \rho_u\, \zeta_t + \sigma_u\, \varepsilon_{t+1}^u, \tag{61}$$

$$\mu_{t+1} = \rho_v\, \mu_t + \sigma_v\, \varepsilon_{t+1}^v. \tag{62}$$

6.3 Steady State

At the deterministic steady state, all shocks are zero ($a = \zeta = \mu = 0$, $\mathrm{rp} = 0$) and variables are constant. From the Euler equation (51): $\beta(1 + r_w) = 1$, which pins down β. The remaining values are determined sequentially:

1. From the production FOCs (labour and capital):

$$A_{\text{coeff}} \equiv \left(\frac{\alpha}{r_w + \delta}\right)^{\alpha/(1-\alpha)}, \qquad \bar{H} = \left((1-\alpha)\, A_{\text{coeff}}\right)^{1/(\omega-1)}.$$

2. Output, capital, investment, consumption:

$$\bar{Y} = A_{\text{coeff}}\, \bar{H}, \quad \bar{K} = \frac{\alpha}{r_w + \delta}\, \bar{Y}, \quad \bar{I} = \delta\, \bar{K}, \quad \bar{C} = \bar{Y} - \bar{I} - r_w\, \bar{d}.$$

3. Marginal utility and ratios:

$$\bar{\Lambda} = (\bar{C} - \bar{H}^{\omega}/\omega)^{-\gamma}, \qquad \overline{\mathrm{tb}} = 1 - (\bar{C} + \bar{I})/\bar{Y}, \qquad \overline{\mathrm{ca}} = 0.$$

With the calibration above, the steady-state values (in the model's internal representation) are:

$d = \bar{d}$	Debt (level)	0.7442
$k = \log \bar{K}$	Capital (log)	1.2231
$r = \log r_w$	Interest rate (log)	−3.2189
$c = \log \bar{C}$	Consumption (log)	0.1106
$h = \log \bar{H}$	Hours (log)	0.0074
$\mathrm{GDP} = \log \bar{Y}$	Output (log)	0.3964
$i = \log \bar{I}$	Investment (log)	−1.0795
$\lambda = \log \bar{\Lambda}$	Marginal utility (log)	1.7244
tb	Trade balance/GDP	0.0200

6.4 Second-Order Perturbation: Why $\|H_{xx}\|$ and $\|G_{xx}\|$ Are Large

The second-order perturbation of the SGU model produces coefficients with Frobenius norms $\|H_{xx}\| \approx 604$ and $\|G_{xx}\| \approx 2{,}731$, orders of magnitude larger than for the 3-equation NK model (where both are essentially zero) or the simple test models in SGU Schmitt-Grohé and Uribe (2004) (where $\|H_{xx}\| < 0.25$). This subsection explains why.

6.4.1 Comparison with SGU (2004)

The paper "Solving dynamic general equilibrium models using a second-order approximation to the policy function" by SGU Schmitt-Grohé and Uribe (2004) describes the *second-order perturbation method* and tests it on three simple models:

Model	n_x	n_y	**Normalization**	$\|H_{xx}\|$
Neoclassical growth	2	1	All log	~ 0.08
Two-country RBC	4	1	All log	~ 0.22
Asset pricing	1	1	All log	~ 0.42
SGU small open economy	7	8	Mixed log/level	~ 604

The SGU small open economy model is a *different model* from any in SGU Schmitt-Grohé and Uribe (2004)—it originates from SGU Schmitt-Grohé and Uribe (2003) ("Closing small open economy models"). Its much larger second-order terms arise from three reinforcing mechanisms.

6.4.2 Mechanism 1: The Hessian Tensor Ψ

The Hessian tensor $\Psi_{i,a,b} = \partial^2 H_i / \partial v_a \partial v_b$ (where $v = (y', y, x', x)$) measures the curvature of the equilibrium conditions at the steady state. For the SGU model, $\Psi \in \mathbb{R}^{15\times 30\times 30}$ with $\|\Psi\| = 631$.

The curvature is concentrated in two equations:

Equation	$\|\Psi_i\|$	**Source of curvature**
Eq. 5: marginal utility (λ)	459	$(C - H^\omega/\omega)^{-\gamma}$ with $\gamma = 2$
Eq. 6: labour FOC (k^f)	432	$H^{\omega-1}(C - H^\omega/\omega)^{-\gamma}$
All other equations	< 14 each	

The GHH preference structure $(C - H^\omega/\omega)^{-\gamma}$ is the dominant source. Taking second derivatives yields terms proportional to $\gamma(\gamma+1)(C - H^\omega/\omega)^{-\gamma-2}$, which with $\gamma = 2$ and $\bar{C} - \bar{H}^\omega/\omega \approx 0.44$ gives a factor of ~ 82 per derivative pair. The largest individual entries are:

Entry	**Equation**	**Value**
$\Psi[\lambda, h, h]$	Marginal utility	253
$\Psi[k^f, h, h]$	Labour FOC	232
$\Psi[\lambda, c, h]$	Marginal utility	-227
$\Psi[\lambda, c, c]$	Marginal utility	207
$\Psi[k^f, c, c]$	Labour FOC	206

For comparison, the 3-equation NK model has $\|\Psi\| = 3.17$ with all curvature concentrated in the Euler equation (from $e^{-\sigma_c \hat{y}}$), and the curvature involves only controls, not states—hence $M^T \Psi M \approx 0$ and $H_{xx} \approx 0$.

6.4.3 Mechanism 2: Near-Unit-Root Dynamics and Sylvester Amplification

The second-order coefficients H_{xx} and G_{xx} satisfy the Sylvester equation (A.5) of Childers et al. Childers et al. (2024):

$$A \cdot X \cdot (h_x \otimes h_x)^T + D \cdot X = -C,$$

which vectorizes to $(F^T \otimes A + I \otimes D)\,\mathrm{vec}(X) = -\mathrm{vec}(C)$.

The left-hand side matrix LHS $\in \mathbb{R}^{735\times 735}$ has condition number $\kappa(\text{LHS}) = 3.24 \times 10^8$, meaning it amplifies the already-large right-hand side $\|C\| = 1{,}493$ by up to 10^8. The ill-conditioning stems from the eigenvalues of h_x:

Eigenvalue	$\lvert\lambda\rvert$
$\lambda_0 = 0.9967$	$\leftarrow$ debt dynamics (near unit root)
$\lambda_1 = 0.478$	capital
$\lambda_2 = 0.420$	risk premium
$\lambda_3 = 0.400$	preference shock
$\lambda_4 = 0.200$	TFP
$\lambda_5, \lambda_6 \approx 0$	

The product $\lambda_0 \cdot \lambda_0 = 0.993$ is very close to 1, making the denominator of the Sylvester solution near-singular for the (d, d) block. This is the "debt eigenvalue" problem: the elastic interest rate closure with small $\psi = 7.42 \times 10^{-4}$ makes debt dynamics persistent ($|\lambda| \to 1$), which amplifies the second-order debt terms.

6.4.4 Mechanism 3: Mixed Log/Level Normalization

In fully log-linearized models, steady-state values drop out of the Hessian because $\partial^2/\partial(\log X)^2 \propto 1$ regardless of $\bar{X}$. In the SGU model, debt d is in levels while capital k is in logs. When the equilibrium conditions combine level and log variables through $e^{(\cdot)}$ terms, the Hessian picks up scale factors from the steady-state values. For example, the resource constraint (48) has $(1 + e^r)d$ with $e^{\bar{r}} = r_w = 0.04$ and $\bar{d} = 0.7442$, creating cross-derivative terms proportional to these level values.

6.4.5 Decomposition of H_{xx} and G_{xx} by Row

The second-order coefficients are concentrated in specific rows:

H_{xx} **row**	$\|H_{xx}[i,:]\|$	**Dominant entry**
r (interest rate)	601	$H_{xx}[r,(\zeta,\zeta)] = -601$
d (debt)	67	$H_{xx}[d,(\zeta,\zeta)] = 66$
k (capital)	13	$H_{xx}[k,(\zeta,\zeta)] = -12$
All others	< 1	

G_{xx} **row**	$\|G_{xx}[i,:]\|$	**Dominant entry**
i (investment)	2,729	$G_{xx}[i,(\zeta,\zeta)] = -2{,}665$
tb (trade balance)	54	$G_{xx}[\text{tb},(\zeta,\zeta)] = -45$
ca (current account)	54	$G_{xx}[\text{ca},(\zeta,\zeta)] = 45$
All others	< 13	

The preference shock ζ (risk premium shock) dominates both H_{xx} and G_{xx}. This occurs because ζ enters the interest rate (55)–(56) additively (in levels), then interacts with debt dynamics through the Euler equation and resource constraint. The near-unit-root debt eigenvalue amplifies these interactions through the Sylvester equation.

Volatility corrections. Despite the large H_{xx} and G_{xx}, the volatility corrections h_{ss} and g_{ss} are moderate: $\|h_{ss}\| = 0.11$, $\|g_{ss}\| = 0.58$. The largest entry is $g_{ss}[\lambda] = 0.56$ (the precautionary savings motive shifting marginal utility) and $h_{ss}[d] = -0.11$ (debt adjusts downward under uncertainty). These are solved from a separate linear system (A.7) and are not amplified by the Sylvester equation.

6.4.6 Implications for Filtering and Estimation

With $\|H_{xx}\| \approx 604$, even a modest state estimation error of $\|e_x\| \sim 0.01$ in x^f produces a Kronecker correction of order $H_{xx} \cdot (e_x \otimes e_x) \sim 604 \times 10^{-4} \approx 0.06$, which is comparable to the volatility correction h_{ss}. Larger errors (as inevitably arise in the first few filter steps) create much larger corrections, driving x^s to the clipping boundary.

This is a fundamental limitation of all Kalman-type filters—linear KF, EKF, UKF, and pruned variants—for the SGU model at second order. Two alternatives exist:

1. **Particle filter (sequential Monte Carlo)**: Sample from the proposal distribution and reweight, avoiding the Gaussian approximation.

2. **Joint estimation**: Treat all exogenous shocks $\{\varepsilon_t\}_{t=1}^T$ as latent HMC parameters alongside the structural parameters θ, using forward simulation instead of filtering. This is the approach taken by Childers et al. Childers et al. (2024) (see Section 6.5).

6.5 Joint Estimation: The HMCExamples.jl Approach

The Julia package `HMCExamples.jl` by Childers et al. Childers et al. (2024) implements three estimation modes for the SGU model:

Mode	Filter	Order	HMC params
`sgu_1_kalman`	Kalman filter	1st	7 (structural only)
`sgu_1_joint`	Forward simulation	1st	$7 + n_\varepsilon T$
`sgu_2_joint`	Forward simulation	2nd	$7 + n_\varepsilon T$

6.5.1 Why Joint Estimation Avoids the H_{xx} Problem

In joint estimation, all $T \times n_\varepsilon$ shock realizations are drawn as HMC parameters:

$$\varepsilon_{\text{draw}} \sim \mathcal{N}(0, I_{n_\varepsilon T}), \qquad \varepsilon = \text{reshape}(\varepsilon_{\text{draw}},\ n_\varepsilon \times T).$$

With $T = 200$ and $n_\varepsilon = 3$, this gives $7 + 600 = 607$ total HMC parameters (compared to 7 for the Kalman approach).

The log-likelihood is computed by *forward simulation* through the pruned second-order dynamics:

$$x_t^f = h_x\, x_{t-1}^f + \eta\, \varepsilon_t, \tag{63}$$

$$x_t^s = h_x\, x_{t-1}^s + \tfrac{1}{2} H_{xx}(x_{t-1}^f \otimes x_{t-1}^f) + \tfrac{1}{2} h_{ss}, \tag{64}$$

$$\hat{y}_t = g_x(x_t^f + x_t^s) + \tfrac{1}{2} G_{xx}(x_t^f \otimes x_t^f) + \tfrac{1}{2} g_{ss}, \tag{65}$$

and the observation log-density is $\sum_t \log p(y_t^{\text{obs}} \mid \hat{y}_t)$.

The key insight is that with *known* shocks ε_t, the states x_t^f are computed *exactly*—there is no estimation error. The Kronecker product $x^f \otimes x^f$ is evaluated at the true (proposed) state, not at a filtered estimate, so the large H_{xx} does not amplify any estimation uncertainty. The only "error" is the sampling noise from HMC, which is controlled by the leapfrog integrator's step size and the prior on ε.

6.5.2 Dimension and Computational Cost

The joint approach increases the HMC parameter dimension from 7 to 607, which:

- Requires more leapfrog steps per HMC iteration (dimension-dependent mixing);
- Needs more warmup iterations for the mass matrix adaptation;
- Benefits from the differentiable solver (gradients through the pruned dynamics are cheap via automatic differentiation).

Despite the higher dimension, Childers et al. Childers et al. (2024) report successful estimation with the NUTS sampler. The gradient computation is efficient because the forward simulation is a simple scan (sequential matrix multiplications and Kronecker products) with $O(T \cdot n_x^2)$ cost per evaluation.

6.5.3 Implications for This Codebase

The current codebase supports Kalman-type filtering only. Adding a joint estimation mode would require:

1. Augmenting the HMC parameter vector with $n_\varepsilon \times T$ shock parameters.
2. Replacing the filter's `log_likelihood` with a forward-simulation scan.
3. Adding a standard normal prior $\varepsilon \sim \mathcal{N}(0, I)$ on the shock parameters.
4. Adapting the mass matrix for the mixed (structural + shock) parameter space.

This is architecturally straightforward but would require changes to the estimator interface. The approach would enable second-order SGU estimation, which is not currently possible with any filter in the library.

Part II
Solution Methods

7 The Policy Function Concept

A policy function maps the current state and structural parameters to optimal controls:

$$\Pi : \mathbb{R}^2 \times \mathbb{R}^9 \to \mathbb{R}^2, \qquad ((a_t, v_t), \boldsymbol{\theta}) \mapsto (y_t, \pi_t). \tag{66}$$

The interest rate i_t is computed analytically from (44). The policy must satisfy the Euler equations in expectation for all states and parameters:

$$\mathbb{E}_\varepsilon \left[\begin{pmatrix} R_{\text{IS}} \\ R_{\text{NKPC}} \end{pmatrix} \right] = \mathbf{0}. \tag{67}$$

8 First-Order Perturbation Solution

8.1 Method of Undetermined Coefficients

We guess $y_t = g_{ya}\, a_t + g_{yv}\, v_t$, $\pi_t = g_{\pi a}\, a_t + g_{\pi v}\, v_t$ and solve.

The characteristic denominator:

$$\boxed{D(r) = (1-r) + \frac{\phi_y}{\sigma_c} + \frac{\kappa(\phi_\pi - r)}{\sigma_c(1-\beta r)}.} \tag{68}$$

The solutions:

$$g_{ya} = \frac{1-\rho_a}{D(\rho_a)}, \tag{69}$$

$$g_{yv} = \frac{-1/\sigma_c}{D(\rho_v)}, \tag{70}$$

$$g_{\pi a} = \frac{\kappa\, g_{ya}}{1-\beta\rho_a}, \quad g_{\pi v} = \frac{\kappa\, g_{yv}}{1-\beta\rho_v}, \tag{71}$$

$$g_{ia} = \phi_\pi g_{\pi a} + \phi_y g_{ya}, \quad g_{iv} = \phi_\pi g_{\pi v} + \phi_y g_{yv} + 1. \tag{72}$$

The first-order solution is $c_t = g_x\, x_t$ where $g_x \in \mathbb{R}^{3\times 2}$.

9 Second-Order Perturbation Solution

$$c_t = g_x\, x_t + \frac{1}{2} G_{xx}(x_t \otimes x_t) + \frac{1}{2} g_{ss}. \tag{73}$$

The `SmallNKModel._second_order` method uses an ad-hoc parameterisation controlled by the `nl_scale` attribute (set at construction time via `SmallNKModel(nl_scale=2.0)`). At `nl_scale = 0`, the model is exactly linear (all second-order terms are zero). At `nl_scale = 1.0`, the terms correspond to the ad-hoc second-order approximation. At `nl_scale = 2.0` (used in the tutorial), the second-order effects are doubled for testing purposes.

Pitfall: The second-order terms in `SmallNKModel` are *not* derived from the full second-order perturbation equations (which require solving a Sylvester equation). Instead, they use a simplified "ad-hoc" parameterisation where the quadratic coefficients are proportional to products of the first-order coefficients, scaled by `nl_scale * kappa`. For the small NK model with moderate `nl_scale`, this approximation is adequate for testing the neural solver.

The `EpsteinZinNKModel` class provides a more principled second-order solution where all corrections scale with $\delta = \gamma - 1/\psi$, including cross-state interaction terms.

10 The Neural Network Solver

10.1 Architecture

The neural network $\Pi_\psi : \mathbb{R}^{n_s+n_\theta} \to \mathbb{R}^{n_c}$ takes normalised (states, $\tilde{\boldsymbol{\theta}}$) as input and outputs (y, π). The `ScalablePolicyNetwork` auto-selects architecture based on d_{in}:

d_{in} range	Hidden layers	Residual connections
< 20	$[128, 128]$	No
20–59	$[256, 256, 128]$	No
≥ 60	$[512, 256, 256, 256]$	Gated skip

For our NK model ($d_{\text{in}} = 11$, being 2 states + 9 parameters), this gives 128–128 hidden layers. Default activation: tanh (C^∞). The architecture also supports SIREN ($\sin(\omega_0 z)$) and MRePU ($z(z+1)^k \mathbf{1}_{z\geq -1}$) activations, as well as optional spectral normalisation on hidden layers.

The input normalisation is:

$$\tilde{x}_j = \frac{x_j - \mu_j}{s_j}, \tag{74}$$

where μ_j and s_j are the mean and scale of dimension j's training domain. For states, $\mu = 0$ and $s =$ domain half-width. For parameters (or canonical ψ), $\mu = (\psi_{\min} + \psi_{\max})/2$ and $s = (\psi_{\max} - \psi_{\min})/2$, computed from the training domain bounds.

10.2 Canonical Reparameterisation

The 9 structural parameters have vastly different posterior spreads and correlations. For the test model, the posterior covariance condition number is approximately 1150. Training the neural network directly in $\boldsymbol{\theta}$-space means that gradient penalties (Sobolev, PG) are dominated by loosely-constrained parameters, while tightly-constrained parameters—where HMC divergences originate—are under-regularised.

Canonical reparameterisation addresses this by transforming to a decorrelated, unit-scale space. Given preliminary posterior samples (from a quick perturbation-based HMC), we compute the posterior covariance Σ_θ and its eigendecomposition square root:

$$M = V\Lambda^{1/2}, \qquad \psi = M^{-1}(\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{mode}}), \tag{75}$$

where $V\Lambda V^\top = \Sigma_\theta$. In ψ-space, parameters are decorrelated and approximately unit-variance.

The neural network is trained entirely in ψ-space: the NN input receives $\psi_{\text{norm}} = (\psi - \mu_\psi)/\sigma_\psi$; training samples are drawn in ψ-space, round-tripped through $\boldsymbol{\theta}$ for bound enforcement; Euler equation residuals internally convert $\psi \to \boldsymbol{\theta}$ for economic consistency; and the external interface (`solve(θ)`) accepts structural $\boldsymbol{\theta}$ and converts automatically.

The impact is dramatic. Without any explicit gradient regularisation, reparameterisation alone reduces divergences from 129 to 4 (a 32-fold improvement). With Sobolev regularisation, reparameterisation turns a FAIL (6 divergences) into a PASS (0 divergences, $\hat{R} = 1.035$, ESS $= 255$). With PG regularisation, the optimal λ_{pg} decreases from 5×10^{-2} (in $\boldsymbol{\theta}$-space) to 10^{-2} (in ψ-space), achieving 0 divergences, $\hat{R} = 1.018$, ESS $= 283$.

The mechanism is clear: in ψ-space, the Frobenius penalty $\|\partial R/\partial\psi\|_F^2$ equally weights all decorrelated parameter directions. In $\boldsymbol{\theta}$-space, the same penalty is dominated by parameters with large $\partial R/\partial\theta$ (typically the loosely-constrained ones), leaving the tightly-constrained directions under-regularised.

Pitfall: The preliminary posterior samples used to compute M must be obtained from a separate estimation run (e.g., using the perturbation model). In the tutorial, Stage 1 runs a short single-chain HMC with the perturbation model to generate these samples. If the preliminary posterior is very different from the true posterior (e.g., due to severe model misspecification), the reparameterisation may be suboptimal but is unlikely to be harmful.

10.3 The All-in-One (AiO) Trick

Proposition 1 (AiO identity)**.** *For two independent shock draws $\varepsilon^{(1)} \perp \varepsilon^{(2)}$:*

$$\mathbb{E}_{\varepsilon^{(1)},\varepsilon^{(2)}}\left[R^{(1)} \cdot R^{(2)}\right] = \left(\mathbb{E}_\varepsilon[R]\right)^2. \tag{76}$$

10.4 AiO Loss with Residual Balancing

$$\mathcal{L}_{\text{AiO}} = \sum_{j=1}^{2}\left[\exp(-\ell_j) \cdot \frac{1}{B}\sum_{b=1}^{B} R_j^{(1),b} R_j^{(2),b} + \ell_j\right]. \tag{77}$$

The learnable per-equation log-variance weights ℓ_j implement residual balancing à la Kendall et al. (2018). Without clamping, the weights can diverge and one equation dominates. With $\max|\ell_j| \leq 5$ (the `max_log_var` parameter), the weight ratio stays bounded at approximately 148:1.

10.5 Training Pipeline

The total training loss:

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{AiO}}}_{\text{Euler residual}} + \lambda_{\text{sob}} \underbrace{\mathcal{L}_{\text{sob}}}_{\text{Sobolev}} + \lambda_{\text{pg}} \underbrace{\mathcal{L}_{\text{pg}}}_{\text{pert. gradient}} + \lambda_{\text{gc}} \underbrace{\widetilde{\mathcal{L}}}_{\text{grad. consistency}} + \lambda_{\text{wd}} \underbrace{\|\boldsymbol{\psi}\|^2}_{\text{weight decay}} . \tag{78}$$

Under canonical reparameterisation, the recommended R3 configuration uses: $\lambda_{\text{sob}} = 10^{-3}$, $\lambda_{\text{pg}} = 10^{-2}$, $\lambda_{\text{gc}} = 0$, $\lambda_{\text{wd}} = 10^{-3}$, 30,000 epochs. In $\boldsymbol{\theta}$-space (without reparameterisation), the original G9 configuration uses $\lambda_{\text{pg}} = 5 \times 10^{-2}$.

The training pipeline has three phases: a warmup phase (first 25% of epochs) with AiO loss only, then the full loss including gradient penalties for the remaining 75%. This allows the network to first learn approximate function values before being constrained on gradients.

10.6 Supervised Pretraining

Before AiO training, the neural solver is warm-started by training on targets from the 2nd-order perturbation solution. The pretraining uses dimension-dependent defaults (via `pretrain_defaults`), including a pool of parameter vectors drawn from the inflated posterior, cosine learning rate decay, early stopping with patience, curriculum learning (first-order-only targets for an initial phase, then full 2nd-order), and periodic state-grid refresh to prevent overfitting.

Pretraining is *essential*: it initialises the network near the true policy surface, reducing the burden on unsupervised AiO training. Without pretraining, the AiO loss landscape has poor local minima.

10.7 Coefficient Extraction Methods

After training, the neural solver must produce a `ModelSolution` object (containing g_x, G_{xx}, g_{ss}, etc.) for downstream use by the UKF and HMC pipeline. Three methods are available, selected via `solve_method`:

Finite-difference stencil (`solve_fd`). Uses $1 + 2n_s^2$ probe points arranged as central differences and cross-differences. XLA-safe (no GradientTape calls). Best for small state dimensions ($n_s \leq 3$).

Autodiff Jacobian + FD Hessian (`solve_autodiff`). Uses exact autodiff for the Jacobian g_x and central FD of the Jacobian for the Hessian G_{xx}. Cost: $1 + 2n_s$ autodiff probes.

Ridge-regression Hessian (`solve_regression`, default). Uses FD for the Jacobian (same as `solve_fd`) and then fits the Hessian via ridge regression on random samples near the origin. The procedure subtracts the known constant and linear terms from the neural network output, then regresses the residual on a quadratic design matrix. This avoids the numerical instability of h^4 denominators in pure FD. Cost: single batched NN call with $1 + 2n_s + N$ inputs, where $N \approx 1.5n_s^2$. Fully JIT-safe. Scales to $n_s = 30$ (NAWM-class models).

11 Gradient Regularisation

11.1 The Gradient Quality Problem

HMC requires $\nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}|Y)$. A key insight: *a neural network can have very accurate function values but very noisy derivatives.* The policy function might predict $y(\boldsymbol{\theta}) = 0.032$ accurately, but $\partial y/\partial \kappa$ might fluctuate wildly. This is because training minimises a loss involving $\Pi_{\boldsymbol{\psi}}$ but places no direct constraint on $\partial \Pi_{\boldsymbol{\psi}}/\partial \boldsymbol{\theta}$.

The central challenge is therefore: *train* $\Pi_{\boldsymbol{\psi}}$ *such that not only* $\Pi_{\boldsymbol{\psi}} \approx \Pi$*, but also* $\nabla_{\boldsymbol{\theta}}\Pi_{\boldsymbol{\psi}} \approx \nabla_{\boldsymbol{\theta}}\Pi$ *throughout the posterior support.*

Two complementary approaches address this: working in the right parameter space (canonical reparameterisation, Section 10.2), and explicitly regularising the gradients (this section).

11.2 Sobolev Regularisation

$$\mathcal{L}_{\text{sob}} = \mathbb{E}_{x,\boldsymbol{\theta}}\left[\left\|\frac{\partial R}{\partial \boldsymbol{\theta}}\right\|_F^2\right]. \tag{79}$$

Estimated via the double Hutchinson estimator:

$$\|A\|_F^2 = \mathbb{E}_{\mathbf{w},\mathbf{v}}\left[\left(\mathbf{w}^\top A\mathbf{v}\right)^2\right], \tag{80}$$

where $\mathbf{w} \sim \mathcal{N}(0, I_{n_e})$ and $\mathbf{v} \sim \mathcal{N}(0, I_{n_\theta})$. Typically 2 random projections are used per epoch.

In $\boldsymbol{\theta}$-space, Sobolev reduces divergences from ∼129 (no regularisation) to ∼6 but is not sufficient. Under canonical reparameterisation, Sobolev alone achieves 0 divergences (configuration R1: $\hat{R} = 1.035$, ESS = 255). The mechanism: in ψ-space, the Frobenius penalty equally weights all decorrelated parameter directions, whereas in $\boldsymbol{\theta}$-space the penalty is dominated by loosely-constrained parameters.

11.3 Perturbation Gradient Regularisation (PG)

$$\mathcal{L}_{\text{pg}} = \sum_{c=1}^{n_c} \mathbb{E}_{x,\boldsymbol{\theta}}\left\|\frac{\partial \Pi_{\boldsymbol{\psi},c}}{\partial \boldsymbol{\theta}} - \frac{\partial \Pi_{\text{pert},c}}{\partial \boldsymbol{\theta}}\right\|^2. \tag{81}$$

PG provides a *specific, smooth target* for the neural network's parameter gradients, computed from the analytical perturbation solution. The perturbation gradients $\partial\Pi_{\text{pert}}/\partial\boldsymbol{\theta}$ are computed using TensorFlow autodiff through the analytical perturbation formulas implemented in `_perturbation_output_tf`.

PG is the *single most effective gradient regularisation technique* tested. Under canonical reparameterisation with $\lambda_{\text{pg}} = 10^{-2}$, it achieves 0 divergences, $\hat{R} = 1.018$, ESS = 283 (configuration R3).

PG requires a perturbation solution, which is available for standard DSGE models but may be inaccurate for highly nonlinear models or models with occasionally binding constraints. For such models, the finding that Sobolev-only passes under reparameterisation (R1) provides an important fallback.

11.4 Gradient-Consistency Loss (GC)

Derived from the implicit function theorem: differentiating $\mathbb{E}[R] = 0$ with respect to $\boldsymbol{\theta}$ gives $\mathbb{E}[\nabla_{\boldsymbol{\theta}} R] = 0$.

$$\widetilde{\mathcal{L}}(\boldsymbol{\psi}) = \mathbb{E}_{x,\boldsymbol{\theta}}\left[\|\mathbb{E}_\varepsilon[\nabla_{\boldsymbol{\theta}} g]\|^2\right]. \tag{82}$$

Estimated via AiO debiasing: $\widetilde{\mathcal{L}} \approx \mathbb{E}[(\nabla_{\boldsymbol{\theta}} g^{(1)})^\top(\nabla_{\boldsymbol{\theta}} g^{(2)})]$.

GC enforces a *necessary condition* (the policy gradient must satisfy the differentiated Euler equation), whereas PG provides a *sufficient target* (match the known perturbation gradient). A necessary condition has many solutions, while PG pins down the specific smooth solution. Accordingly, GC is consistently weaker than PG in experiments, both in $\boldsymbol{\theta}$-space (G11: 3 divergences) and under reparameterisation (R2: 6 divergences).

GC may be valuable for models where the perturbation approximation is poor, as it provides model-consistent gradient regularisation without relying on an external approximation.

Part III
Statistical Inference

12 State-Space Models and Filtering

12.1 The DSGE State-Space Form

The second-order perturbation solution produces a nonlinear state-space model.

Naive (unpruned) transition:

$$x_t = h_x\, x_{t-1} + \tfrac{1}{2} H_{xx}(x_{t-1} \otimes x_{t-1}) + \tfrac{1}{2} h_{ss} + \eta\, \varepsilon_t. \tag{83}$$

Observation:

$$y_t^{\text{obs}} = g_x\, x_t + \tfrac{1}{2} G_{xx}(x_t \otimes x_t) + \tfrac{1}{2} g_{ss} + e_t, \quad e_t \sim \mathcal{N}(0, R). \tag{84}$$

The instability problem. Using (83) directly for filtering is numerically dangerous for models of moderate or large dimension. The Kronecker product $x_{t-1} \otimes x_{t-1}$ feeds back quadratically into x_t, which then enters the next Kronecker product. This creates explosive variance growth: for the SGU model ($n_s = 7$, $H_{xx} \in \mathbb{R}^{7\times 49}$, $\|H_{xx}\| \approx 604$), the augmented state-space matrix of the equivalent linear filter has large spectral radius, causing immediate numerical divergence. The same problem afflicts both the Augmented Kalman Filter (which expands the state to $[x, \text{vec}(xx'), \tilde{x}]$ of dimension $n_s + n_s^2 + n_s$) and the standard (unpruned) UKF that propagates sigma points through the full nonlinear transition.

12.2 Pruned Second-Order State-Space

The solution to the instability problem is the *pruned* decomposition of Kim, Kim, Schaumburg & Sims Kim et al. (2008) and Andreasen, Fernández-Villaverde & Rubio-Ramírez Andreasen et al. (2018). The state is decomposed into a first-order component x_t^f and a second-order correction x_t^s:

$$x_t^f = h_x\, x_{t-1}^f + \eta\, \varepsilon_t, \tag{85}$$

$$x_t^s = h_x\, x_{t-1}^s + \tfrac{1}{2} H_{xx}(x_{t-1}^f \otimes x_{t-1}^f) + \tfrac{1}{2} h_{ss}. \tag{86}$$

The full state is recovered as $x_t = x_t^f + x_t^s$.

The key insight: only the first-order component x^f enters the Kronecker product in (86). Since x^f evolves linearly and is driven by h_x (whose eigenvalues satisfy $|\lambda_i| < 1$ by the Blanchard–Kahn conditions), the quadratic forcing term $H_{xx}(x^f \otimes x^f)$ remains bounded. The correction x^s is then driven by $h_x x^s$ (stable) plus a bounded forcing term—no explosive self-feedback.

The corresponding pruned observation equation uses $x^f + x^s$ for the linear part but only x^f for the quadratic:

$$y_t^{\text{obs}} = g_x\, (x_t^f + x_t^s) + \tfrac{1}{2} G_{xx}(x_t^f \otimes x_t^f) + \tfrac{1}{2} g_{ss} + e_t. \tag{87}$$

This is exactly the decomposition used in the `simulate` method (`base.py`) and, for models with moderate $\|H_{xx}\|$, in the `PrunedSquareRootUKF` filter class.

12.3 The Kalman Filter

For first-order models ($H_{xx} = 0$, $G_{xx} = 0$), the state-space is linear and the standard Kalman filter is exact.

Predict:

$$\hat{x}_{t|t-1} = h_x\, \hat{x}_{t-1|t-1}, \tag{88}$$
$$P_{t|t-1} = h_x\, P_{t-1|t-1}\, h_x^\top + Q. \tag{89}$$

Update:

$$v_t = y_t^{\text{obs}} - Z\, \hat{x}_{t|t-1}, \tag{90}$$
$$S_t = Z\, P_{t|t-1}\, Z^\top + R, \tag{91}$$
$$K_t = P_{t|t-1}\, Z^\top\, S_t^{-1}, \tag{92}$$
$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t\, v_t, \tag{93}$$
$$P_{t|t} = (I - K_t Z) P_{t|t-1} (I - K_t Z)^\top + K_t R K_t^\top. \tag{94}$$

Log-likelihood:

$$\ell_t = -\frac{n_y}{2} \log(2\pi) - \frac{1}{2} \log |S_t| - \frac{1}{2} v_t^\top S_t^{-1} v_t. \tag{95}$$

The implementation uses the Cholesky factorisation of S_t for numerical stability, computing $\log |S_t| = 2 \sum_i \log L_{ii}$ and the quadratic form via forward substitution. The filter loop is implemented using `b.foldl` (which maps to `tf.foldl`), creating a single traced loop body rather than unrolling T iterations.

12.4 The Square-Root Unscented Kalman Filter

For nonlinear models, the Unscented Kalman Filter (UKF) propagates a set of deterministically chosen sigma points through the nonlinear dynamics, capturing mean and covariance to second order without explicit Jacobian computation Van der Merwe and Wan (2001).

Sigma-point parameters. Given an L-dimensional state, $2L+1$ sigma points are generated with parameters $\alpha = 1$, $\beta_{\text{UKF}} = 2$, $\kappa = \max(0, 3-L)$, giving $\lambda = \alpha^2(L+\kappa) - L$ and $\gamma = \sqrt{L+\lambda}$:

$$\mathcal{X}_0 = \hat{x}, \quad \mathcal{X}_i = \hat{x} + \gamma\, S_{:,i}, \quad \mathcal{X}_{L+i} = \hat{x} - \gamma\, S_{:,i}, \tag{96}$$

where S is the Cholesky factor of the state covariance ($P = SS^\top$). The mean weights are $w_0^m = \lambda/(L+\lambda)$, $w_i^m = 1/(2(L+\lambda))$, and the covariance weights are $w_0^c = w_0^m + 1 - \alpha^2 + \beta_{\text{UKF}}$, $w_i^c = w_i^m$. With $\alpha = 1$, the central covariance weight w_0^c is guaranteed positive.

Square-root formulation. The "square-root" formulation Van der Merwe and Wan (2001) maintains numerical stability by working directly with the Cholesky factor S rather than the full covariance P:

- **Predict:** After propagating sigma points through the transition function, the predicted Cholesky factor is obtained via QR decomposition of the stacked sigma-point residuals and process noise Cholesky factor. Since $w_0^c > 0$ (with $\alpha = 1$), the central sigma point residual can be folded into the QR factorisation rather than requiring a separate rank-1 update.
- **Update:** The measurement update applies sequential rank-1 Cholesky downdates Gill et al. (1974), one for each observation dimension.

General-dimension Cholesky downdate. Given a lower-triangular L with $LL^\top = P$ and a vector v, the rank-1 downdate computes $\bar{L}$ such that $\bar{L}\bar{L}^\top = P - vv^\top$. The implementation uses the standard algorithm of Gill, Golub, Murray & Saunders Gill et al. (1974), implemented as a `tf.while_loop` over column indices $k = 0, \ldots, L-1$. Diagonal and sub-diagonal elements are clamped to a small positive constant (10^{-12}) to prevent NaN when the downdate nearly exhausts the available variance.

The `SquareRootUKF` class implements the unpruned formulation for models where the quadratic terms are small. For models with larger second-order coefficients, the pruned variant described next can be used.

12.5 Pruned Square-Root UKF

The `PrunedSquareRootUKF` class implements a pruned second-order filter following the state-space decomposition of Kim et al. **?** and Andreasen et al. Andreasen et al. (2018). The UKF state is $x_t^f \in \mathbb{R}^{n_s}$ (first-order component); the second-order correction x_t^s is tracked as a deterministic side variable, matching the simulation's data-generating process.

Transition. The first-order component evolves linearly, so the UKF sigma-point predict is exact:

$$\hat{x}^f_{t|t-1} = h_x\, x^f_{t-1|t-1}, \tag{97}$$

$$P^f_{t|t-1} = h_x\, P^f_{t-1|t-1}\, h_x^\top + Q_f, \tag{98}$$

where $Q_f = \eta\eta^\top + \epsilon I$.

Observation (UKF sigma-point update). The observation equation is nonlinear in x^f through the $\mathrm{kron}(x^f, x^f)$ term. Rather than linearising (as in an EKF), sigma points $\{\chi^{(i)}\}$ are generated from $(\hat{x}^f_{t|t-1}, S^f_{t|t-1})$ and propagated through the full observation function:

$$y^{(i)} = g_x(\chi^{(i)} + x_t^s) + \tfrac{1}{2}G_{xx}(\chi^{(i)} \otimes \chi^{(i)}) + \tfrac{1}{2}g_{ss}, \tag{99}$$

where x_t^s is a fixed offset (the same for all sigma points). The predicted observation $\hat{y}_t$, innovation covariance S_y, and cross-covariance P_{xy} are then computed from the sigma-point statistics, and the standard SR-UKF update formulae give $x^f_{t|t}$ and $S^f_{t|t}$.

Deterministic x^s update. After the Kalman update, x^s is updated from the filtered x^f mean:

$$x^s_{t+1} = \mathrm{clip}\Big(h_x\, x_t^s + \tfrac{1}{2}H_{xx}(\hat{x}^f_{t|t} \otimes \hat{x}^f_{t|t}) + \tfrac{1}{2}h_{ss},\ -C,\ C\Big), \tag{100}$$

where $\hat{x}^f_{t|t}$ is the filtered posterior mean and $C = 50$ matches the state clipping in `simulate()`. The point estimate $\hat{x}^f_{t|t} \otimes \hat{x}^f_{t|t}$ (rather than $\mathbb{E}[x^f \otimes x^f] = \hat{x}^f\hat{x}^{f\top} + P^f$) is used to match the simulation's realised dynamics.

Initial conditions. Both x^f and x^s are initialised to zero, with P_0^f set to the steady-state covariance of the first-order system.

Applicability and limitations. The filter requires that $\|H_{xx}\|$ and $\|G_{xx}\|$ be moderate so that Kalman estimation errors in x^f—which are of order $\|P^f\|^{1/2}$—do not get explosively amplified through the Kronecker product:

$$\text{Stability requires } \|H_{xx}\| \cdot \|P_0^f\|^{1/2} \ll 1. \tag{101}$$

For the SmallNK model with `nl_scale` ≤ 5, this condition is satisfied and the pruned UKF agrees closely with the full UKF (relative log-likelihood difference $< 0.01\%$).

For models with very large $\|H_{xx}\|$—notably the SGU model where $\|H_{xx}\| \approx 604$ and $\|G_{xx}\| \approx 2{,}731$ (see Section 6.4)—the Kronecker term amplifies even small estimation errors by a factor of hundreds, causing x^s to saturate at the clipping boundary immediately. The resulting observation bias destabilises the filter within a few steps. This is a fundamental limitation of all Kalman-type filters (EKF, UKF, or pruned variants) for strongly nonlinear second-order models.

Filter	**State dim.**	**Method**	**Applicability**
KalmanFilter	n_s	Linear KF	First-order models
SquareRootUKF	n_s	UKF (sigma pts)	Moderate nonlinearity
PrunedSquareRootUKF	n_s (UKF)	Pruned UKF	Moderate $\|H_{xx}\|$
Particle filter	n_s	Sequential MC	Any (future work)

Following standard practice in the DSGE literature Fernández-Villaverde and Rubio-Ramírez (2007), the SGU tutorials use first-order perturbation for filtering and estimation. For second-order SGU, alternatives include a particle filter (sequential Monte Carlo) or the joint estimation approach of Childers et al. Childers et al. (2024) described in Section 6.5.

12.6 Measurement Noise Specification

The measurement noise covariance R is set as $R = r^2 \cdot I_{n_y}$, where r is the measurement noise standard deviation. In the code, $R = 10^{-4} I_2$ (i.e., $r = 0.01$). This small measurement noise means the filter places almost all weight on the observations.

13 Bayesian Estimation

13.1 Parameter Transforms and Prior Specification

HMC operates in unconstrained space $u \in \mathbb{R}^9$. Each parameter θ_j is obtained from u_j through a *bijective transform* $\theta_j = T_j(u_j)$ chosen so that θ_j always lies in its admissible domain. The transforms are defined in `SmallNKModel.transform_from_unconstrained` and are: scaled sigmoid for β; softplus for κ, σ_c, ϕ_y, σ_a, σ_v; shifted softplus for ϕ_π; and tanh for ρ_a, ρ_v.

The prior in unconstrained space is:

$$p(u) = \prod_{j=0}^{8} \mathcal{N}(u_j \mid 0, \sigma^2_{\text{prior}}), \qquad \sigma_{\text{prior}} = 2. \tag{102}$$

The total log-posterior (the function whose gradient HMC requires) is:

$$\log p(u|Y) = \underbrace{\log p(Y|\boldsymbol{\theta}(u))}_{\text{log-likelihood from filter}} + \underbrace{\sum_{j=0}^{8} \log \mathcal{N}(u_j|0, \sigma^2_{\text{prior}})}_{\text{log-prior in } u\text{-space}} + \text{const.} \tag{103}$$

There is no explicit Jacobian correction term because HMC operates entirely in u-space. The transform $\boldsymbol{\theta}(u)$ is embedded inside the likelihood: the code calls `transform_from_unconstrained(u)` to get $\boldsymbol{\theta}$, then computes the likelihood given $\boldsymbol{\theta}$.

13.2 Hamiltonian Monte Carlo

Hamiltonian:

$$H(u, p) = -\log p(u|Y) + \frac{1}{2} p^\top M^{-1} p. \tag{104}$$

13.2.1 The Leapfrog Integrator

$$p_{t+1/2} = p_t + \frac{\epsilon}{2}\nabla_u \log p(u_t|Y), \tag{105}$$

$$u_{t+1} = u_t + \epsilon\, M^{-1} p_{t+1/2}, \tag{106}$$

$$p_{t+1} = p_{t+1/2} + \frac{\epsilon}{2}\nabla_u \log p(u_{t+1}|Y). \tag{107}$$

The leapfrog integrator is *symplectic* (preserves volume) and *time-reversible*, with energy error $O(\epsilon^2)$.

13.2.2 Divergent Transitions

A *divergent transition* occurs when the energy error $|\Delta H|$ exceeds a threshold, indicating that the leapfrog integrator encountered a region of very high curvature. In the code, the threshold is applied to the log-acceptance ratio: samples where $\log \alpha < -1000$ are flagged as divergent. Divergent transitions are always rejected but counted for diagnostics.

The fundamental mechanism: the Hessian of the log-posterior involves second derivatives of the neural policy with respect to $\boldsymbol{\theta}$. If these are irregular, the curvature changes discontinuously along the leapfrog trajectory, causing energy divergence. This is why gradient regularisation targets the *smoothness* of $\partial\Pi/\partial\boldsymbol{\theta}$, not just the accuracy of Π itself.

13.2.3 Three-Phase HMC Pipeline (`MultiChainHMC`)

The `MultiChainHMC` estimator implements a robust three-phase pipeline using TensorFlow Probability's adaptive HMC kernels:

Phase 1 (Probe): Small leapfrog count ($L = 5$), identity mass matrix. Dual averaging adapts the step size ϵ. Samples are used to estimate preconditioning (offset and scale for each parameter).

Phase 2 (Tune): Reparametrise to $z = (u - \text{offset})/\text{scale}$. Small leapfrog count ($L = 10$), dual averaging restarts in z-space. The adapted step size ϵ_z determines the production leapfrog count:

$$L_{\text{prod}} = \text{clip}\left(\left\lceil \frac{\pi}{\epsilon_z} \right\rceil, L_{\min}, L_{\max}\right). \tag{108}$$

The target trajectory length is π (a half-period in preconditioned space where the target $\approx \mathcal{N}(0, I)$), capped at `max_leapfrog`.

Phase 3 (Sample): Production draws with adapted ϵ_z and L_{prod}. Tracks divergences per chain. Samples are transformed back to u-space and then to $\boldsymbol{\theta}$-space.

The pipeline includes safeguards: per-chain garbage collection, step-size retry if acceptance drops below 10%, adaptive L from the adapted step size, and separate XLA compilations per phase (small L for warmup keeps compilation fast).

13.3 MCMC Diagnostics

Convergence is assessed using three diagnostics following Vehtari et al. (2021):

Split-$\hat{R}$. Each chain of N samples is split into two halves, giving $2 \times n_{\text{chains}}$ half-chains. $\hat{R}$ compares within-half-chain variance to between-half-chain variance, computed on rank-normalised samples. The criterion $\hat{R} < 1.01$ indicates convergence (a stricter threshold than the traditional 1.05).

Bulk and Tail ESS. Bulk ESS measures effective sample size for location (from rank-normalised draws). Tail ESS measures ESS for tails (from folded rank-normalised draws). Both use the initial positive sequence estimator for autocorrelation. The criterion ESS > 400 per parameter indicates sufficient samples.

Divergent transitions. Zero divergences is the target. Any divergence indicates potential bias.

13.4 Pass/Fail Criteria

A configuration **passes** if all three hold:

1. 0 divergent transitions,
2. $\hat{R} < 1.05$ for all parameters,
3. Minimum bulk ESS > 50.

Part IV
Experiments and Results

14 Complete Experimental Protocol

14.1 Data Generation

Setting	Value	Code location
True parameters $\boldsymbol{\theta}_{\text{true}}$	Eq. (1)	`tutorial: THETA_TRUE`
Sample length T	**200**	`tutorial: T_OBS = 200`
Nonlinearity scale	**2.0**	`tutorial: NL_SCALE=2.0`
Observed variables	(y_t, π_t)	`obs_ix = [0, 1]`
Random seed (Cases A, B)	**42**	`seed=42`
Random seed (Case C data)	**99**	`seed=99`

Cases A and B use data generated with `seed=42`; Case C uses `seed=99` to avoid look-ahead bias.

14.2 Neural Network Training: Full Hyperparameters

Hyperparameter	Symbol	Value (R3)	Code location
Architecture			
Hidden layers	L	2	`ScalablePolicyNetwork`
Hidden width	d_h	128	`ScalablePolicyNetwork`
Activation	ϕ	tanh	`ScalablePolicyNetwork`
Input dimension	d_{in}	11 (2 states + 9 params)	
Output dimension	d_{out}	2 (y, π)	
Total parameters	$\lvert\boldsymbol{\psi}\rvert$	∼18,000	
Reparameterisation		**True** (default)	`reparam=True`
Pretraining (supervised)			
Pretraining epochs		dimension-dependent	`pretrain_defaults`
Pretraining LR		3×10^{-3} (cosine decay)	
Pool size	n_{pool}	500	`n_pool`
LR schedule		cosine decay to 0	
Early stopping patience		dimension-dependent	
AiO training			
AiO epochs		**30,000**	`epochs`
Initial LR	$\alpha_{\max}$	10^{-3}	`lr`
LR schedule		cosine decay to 10^{-6}	
Batch size (states)	B	**512**	`batch`
Warmup fraction		0.25	`warmup_frac`
Gradient clip norm	$G_{\max}$	**10**	
Loss weights (R3, reparameterised)			
AiO weight		1 (fixed)	
Sobolev weight	λ_{sob}	10^{-3}	`lam_sob`
PG weight	λ_{pg}	10^{-2}	`lam_pg`
GC weight	λ_{gc}	0	`lam_gc`
Weight decay	λ_{wd}	10^{-3}	`weight_decay`
Residual balancing			
Initial ℓ_j		0 (both)	`log_var`
ℓ_j clamp range		$[-5, 5]$	`max_log_var=5.0`

14.3 HMC Settings

Setting	Value	Code location	Notes
Number of chains	**4**	`MultiChainHMC: n_chains`	
Warmup samples	**500**	`num_warmup`	Split across 3 phases
Production samples	**500**	`num_results`	Per chain
Target acceptance rate	0.65	`target_accept`	
Initial step size	0.01	`init_step_size`	
Max leapfrog	auto or 500	`max_leapfrog`	
Prior σ_{prior}	2.0		

15 Experimental Results

15.1 Results Summary

Table 1: Selected results from 30 configurations. Div = divergent transitions, $\hat{R}$ = worst split-$\hat{R}$, ESS = minimum bulk ESS. Configurations prefixed R use canonical reparameterisation (ψ-space). Each failed configuration is analysed in Section 16.

ID	Key Feature	Div	$\hat{R}$	ESS	Time	Status
$\boldsymbol{\theta}$-space (no reparameterisation)						
F1	Sob+Cos+WD, 30k	6	1.058	57	239s	FAIL
F3	No Sobolev	129	1.444	8	167s	FAIL
G2	MRePU_2 activation	4	1.125	20	—	FAIL
G3	SIREN ($\omega_0 = 30$)	2	2.038	—	—	FAIL
G5	Pert-corrected arch	0	1.494	7	—	FAIL
G6	PG ($\lambda = 10^{-2}$)	3	1.044	90	—	FAIL
G9	**PG ($\lambda = 5 \times 10^{-2}$)**	**0**	**1.013**	**198**	**240s**	**PASS**
G11	GC ($\lambda = 5 \times 10^{-2}$)	3	1.017	113	240s	FAIL
G13	GC + PG combined	6	1.292	10	—	FAIL
G14	Spectral norm + PG	5	1.102	34	—	FAIL
G15	DCGD + PG	13	1.040	92	274s	FAIL
G17	Scale-weighted PG	8	1.095	43	—	FAIL
G19	PG + NormGrad	0	1.037	120	261s	PASS
ψ-space (canonical reparameterisation)						
R0	No regularisation + reparam	4	1.069	34	293s	FAIL
R1	**Sob-only + reparam**	**0**	**1.035**	**255**	**365s**	**PASS**
R2	GC + reparam	6	1.035	168	326s	FAIL
R3	**PG (10^{-2}) + reparam**	**0**	**1.018**	**283**	**312s**	**PASS**
R4	PG+GC + reparam	9	1.025	152	376s	FAIL
R5	DCGD+PG + reparam	6	1.137	18	326s	FAIL
R6	Pert-corr + reparam	84	1.307	10	343s	FAIL
R7	PG + normgrad + reparam	0	1.037	65	328s	PASS

15.2 Impact of Canonical Reparameterisation

The reparameterisation results reveal that the gradient quality problem is primarily a *parameter-space geometry* issue, not a network expressiveness issue. Configuration R0 (no regularisation, reparam only) reduces divergences from 129 to 4—a 32-fold improvement from changing only the coordinate system.

The most striking finding is R1: Sobolev-only + reparam achieves 0 divergences and passes all diagnostics, whereas the same Sobolev-only configuration in $\boldsymbol{\theta}$-space (F1) had 6 divergences and failed. This means neural DSGE solvers may be viable even for models without a perturbation solution, using only the model-free Sobolev penalty combined with canonical reparameterisation.

Under reparameterisation, the optimal PG weight decreases from 5×10^{-2} to 10^{-2}. This is expected: in ψ-space the PG penalty is already well-scaled across parameters, so less regularisation is needed.

15.3 Multi-Seed Robustness

A 3-seed comparison found:

Solver	Div	$\hat{\mathbf{R}}$	ESS	RMSE
$\boldsymbol{\theta}$-space, PG 10^{-2}	45 ± 31	1.035 ± 0.022	150 ± 85	0.295 ± 0.009
ψ-space, PG 10^{-2}	$\mathbf{5 \pm 4}$	$\mathbf{1.013 \pm 0.007}$	$\mathbf{630 \pm 533}$	$\mathbf{0.247 \pm 0.002}$

The reparameterised solver achieves a 9× reduction in divergences, significantly better chain convergence, and 6× lower RMSE variability.

15.4 GPU vs CPU Training

GPU training produces 38 divergences for the same G9 configuration that achieves 0 on CPU. The cause is subtle float64 rounding differences in GPU arithmetic (different operation ordering, fused multiply-add) that accumulate over 30,000 epochs, producing a slightly different learned policy whose gradient landscape has sharper features.

For small models, CPU is preferred. The GPU infrastructure (the `migrate_to_cpu()` method) is implemented for scaling to larger models where GPU parallelism can offset the precision cost.

16 Detailed Analysis of Failed Approaches

This section provides a self-contained description of every non-trivial approach that was tested and failed, explaining each algorithm from first principles and precisely identifying the mechanism by which it fails. Where applicable, we note whether the conclusion changed under canonical reparameterisation.

16.1 Dual-Cone Gradient Descent (DCGD)

DCGD (Yu et al., 2020) addresses gradient conflict by modifying each task's gradient so that it never increases any other task's loss. For two objectives ($\mathcal{L}_{\text{AiO}}$ and $\mathcal{L}_{\text{grad}}$), DCGD projects the update onto the bisector of the two normalised gradient vectors.

DCGD fails because the loss components are *complementary*, not adversarial: at the true policy function, all losses are simultaneously zero. Any observed "conflict" during training is transient stochastic noise. The projection destroys useful gradient information by removing components that are only momentarily conflicting due to mini-batch noise, and the AiO loss's inherent stochasticity causes the projected gradient to rotate randomly from batch to batch.

Configuration G15 (DCGD + PG) produces 13 divergences in $\boldsymbol{\theta}$-space. Under reparameterisation (R5), DCGD still fails with 6 divergences and $\hat{R} = 1.137$, confirming that the complementary-objective argument holds regardless of parameter space.

16.2 Parameter-Scale-Weighted Gradient Penalties

Weighting the PG penalty by the posterior standard deviation $\hat{\sigma}_j^2$ was expected to focus regularisation on directions HMC explores. This reasoning is backwards: tightly constrained parameters (small $\hat{\sigma}_j$) have high Fisher information and high curvature, making them the directions where HMC is most sensitive to gradient noise. Scale-weighted PG *down-weights* exactly where regularisation is most needed.

Configuration G17 produces 8 divergences. Under canonical reparameterisation, diagonal scaling (`normalise_grad`) becomes redundant (R7 produces identical $\hat{R} = 1.037$ to G19) because the full-covariance reparameterisation already provides the optimal scaling and decorrelation. The original failure of diagonal normalisation was due to ignoring correlations and using the wrong scaling direction, not due to normalisation being inherently harmful.

16.3 SIREN (Sinusoidal Representation Networks)

SIREN uses $\sin(\omega_0 z)$ activations. It fails for two reasons: spectral bias mismatch (the true policy is nearly linear, but SIREN's oscillatory basis must cancel oscillations through careful interference, making the cancellation fragile) and gradient amplification (the ω_0 factor multiplies every Jacobian, amplifying derivative errors by $O(\omega_0)$ relative to function errors).

Configuration G3 ($\omega_0 = 30$) produces 2 divergences but catastrophic mixing ($\hat{R} = 2.038$). The sinusoidal basis creates oscillatory gradients that corrupt the entire posterior landscape, causing HMC to mix extremely slowly.

16.4 MRePU (Multiply ReLU Power Units)

MRePU_2 uses $\phi(z) = \max(0, z)^2$, which is C^1 but not C^2. The second-derivative discontinuity at $z = 0$ propagates through the chain rule to create discontinuities in the Hessian of the log-posterior, which the leapfrog integrator cannot handle with a fixed step size.

Configuration G2 produces 4 divergences. For gradient-based MCMC, C^∞ activations (tanh) are strongly preferred.

16.5 Spectral Normalisation

Constraining each weight matrix to have spectral norm ≤ 1 bounds the network's maximum sensitivity but is too restrictive: it forces the network to "flatten" its response, preventing faithful representation of the true policy's sensitivity to parameters. Additionally, the Lipschitz bound addresses gradient *magnitude* but not gradient *smoothness*, and the post-update projection can undo training progress.

Configuration G14 produces 5 divergences.

16.6 Perturbation-Corrected Architecture

Learning a correction $\Pi_\psi = \Pi_{\text{pert}} + \delta\Pi_{\text{NN}}$ ensures smooth base gradients, but the correction's $\boldsymbol{\theta}$-gradients are uncontrolled and can oscillate wildly (a function with small values can have large derivatives).

In $\boldsymbol{\theta}$-space, G5 achieves 0 divergences but terrible mixing ($\hat{R} = 1.494$, ESS $= 7$). Under reparameterisation (R6), the architecture *degrades catastrophically* to 84 divergences. The cause is a parameter-space mismatch: the perturbation base operates in $\boldsymbol{\theta}$-space while the NN correction operates in ψ-space, creating gradient artefacts from the additional $\psi \to \boldsymbol{\theta}$ conversion inside the correction path.

Do not combine perturbation-corrected architecture with canonical reparameterisation.

16.7 Combined GC + PG Regularisation

Combining GC and PG produces worse results than PG alone, both in $\boldsymbol{\theta}$-space (G13: 6 divergences) and under reparameterisation (R4: 9 divergences). Three mechanisms: gradient budget competition (both terms compete for the clipped gradient norm), different variance profiles (high-variance GC contaminates low-variance PG), and incompatible optima at finite network capacity.

More regularisation is not always better. When one regulariser (PG) provides a strong, specific, low-variance signal, adding a weaker, noisier regulariser (GC) degrades the result.

16.8 AiO-Only Training Without Regularisation

F3 (no regularisation, $\boldsymbol{\theta}$-space) produces 129 divergences. R0 (no regularisation, ψ-space) produces 4 divergences. The 32-fold improvement from reparameterisation alone confirms that the

gradient quality problem is primarily a parameter-space geometry issue. However, even under reparameterisation, some gradient regularisation is needed to reach zero divergences.

16.9 Summary: Taxonomy of Failure Modes

The failed approaches fall into four categories based on their failure mechanism.

Category 1: Insufficient gradient constraint. F3 (no regularisation) and F1 (Sobolev only, $\boldsymbol{\theta}$-space). The fix is PG regularisation and/or canonical reparameterisation. Under reparameterisation, Sobolev alone becomes sufficient (R1).

Category 2: Misguided gradient modification. DCGD (G15, R5) and scale-weighted PG (G17). These modify the training gradient in harmful ways. Confirmed under reparameterisation.

Category 3: Architectural mismatch. SIREN (G3), MRePU (G2), spectral normalisation (G14), and perturbation-corrected + reparam (R6). The fix is standard tanh architecture without constraints, and not combining perturbation-corrected architecture with reparameterisation.

Category 4: Regularisation interference. Combined GC + PG (G13, R4). Use only the single most effective regulariser (PG).

17 Tutorial Results: Three Approaches Compared

The tutorial (`tutorial_neural_dsge.py`) compares three approaches on $T = 200$ observations from a nonlinear NK model (`nl_scale` = 2.0):

	Case A	**Case B**	**Case C**
DGP	Linear (`nl_scale=0`)	Nonlinear	Nonlinear
Solver	1st-order pert	2nd-order pert	Neural (reparam)
Filter	`KalmanFilter`	`SquareRootUKF`	`SquareRootUKF`

Case C uses a three-stage pipeline: (1) a quick perturbation-based HMC to generate theta-samples for the neural solver's training domain, (2) train the reparameterised neural solver with PG regularisation, and (3) full multi-chain HMC using the neural model. The neural solver is wrapped in a `NeuralSmallNKModel` subclass whose `solve()` method delegates to the trained `ConditionedNeuralNKSolver`.

18 Lessons and Recommendations

18.1 What Works

1. **Canonical reparameterisation** (`reparam=True`, default): the single most impactful architectural decision. Decorrelates parameters and equalises gradient scales, reducing divergences 32-fold even without explicit gradient regularisation. The ∼10% training overhead is negligible.
2. **PG regularisation** ($\lambda = 0.01$ under reparam, $\lambda = 0.05$ in $\boldsymbol{\theta}$-space): the strongest gradient regulariser. Combined with reparameterisation, achieves 0 divergences, $\hat{R} = 1.018$, ESS = 283.
3. **Sobolev regularisation under reparameterisation**: sufficient alone (0 divergences, R1). This broadens applicability to models without perturbation solutions.
4. **Supervised pretraining**: initialises the network near the correct solution. Dimension-dependent defaults with cosine LR, early stopping, and curriculum learning.
5. **Residual balancing with clamped log-variance** ($|\ell_j| \leq 5$): prevents IS/NKPC imbalance.

6. **Cosine LR decay + weight decay**: smooth convergence.
7. tanh **activation**: C^∞ smoothness essential for HMC.

18.2 What Does Not Work

1. **DCGD** (Section 16.1): complementary losses defeat the dual-cone projection. Confirmed under reparameterisation (R5).
2. **Scale-weighted PG** (Section 16.2): superseded by canonical reparameterisation.
3. **SIREN** (Section 16.3): oscillatory basis creates $O(\omega_0)$ gradient amplification.
4. **MRePU** (Section 16.4): C^1-only smoothness creates second-derivative discontinuities.
5. **Spectral normalisation** (Section 16.5): the Lipschitz constraint fights the training gradient.
6. **Perturbation-corrected architecture + reparameterisation** (Section 16.6): parameter-space mismatch causes catastrophic regression (R6: 84 divergences).
7. **Combined GC + PG** (Section 16.7): the noisier GC loss contaminates the cleaner PG signal. Confirmed under reparameterisation (R4).
8. **GPU training for small models**: float64 rounding differences degrade gradient quality.

18.3 Corrected Beliefs from Reparameterisation Experiments

Several conclusions from the original $\boldsymbol{\theta}$-space experiments were revised by the reparameterisation experiments:

"Sobolev regularisation alone is insufficient." Corrected: under reparameterisation, Sobolev alone achieves 0 divergences (R1). The original failure was a symptom of the $\boldsymbol{\theta}$-space geometry, not the Sobolev penalty itself.

"The gradient quality problem is fundamentally about the loss function." Corrected: the 32× improvement from reparameterisation alone (R0) shows that the problem is primarily about parameter-space geometry, with loss function design being secondary.

"Normalising gradient penalties by parameter scale is harmful." Partially corrected: diagonal normalisation (scaling by σ_θ) went in the wrong direction because it ignored correlations. Full covariance reparameterisation ($M = \Sigma^{1/2}$) goes in the right direction and is dramatically effective.

18.4 Scaling to Large Models

For models like the ECB's NAWM ($n_s \sim 30$, $n_\theta \sim 80$):

1. Use `ScalablePolicyNetwork` auto-architecture with gated residual connections.
2. **Always enable canonical reparameterisation** (`reparam=True`). Its value increases with the posterior covariance condition number.
3. Use PG regularisation where perturbation is available; Sobolev + reparam as fallback.
4. Increase pretraining pool to ∼2000 parameter vectors.
5. Use `solve_method='regression'`: scales as $O(n_s^2)$ probes.
6. Train on GPU (`migrate_to_cpu()` before HMC for numerical precision).
7. For covariance estimation with large n_θ, regularise the posterior covariance (shrinkage) if the preliminary sample is small.

19 Complete Code–Equation Cross-Reference

Equation	File	Code location
IS curve, nonlinear (45)	`dsgeneural.py`	`_residuals_tf`, `lhs/R_is1`
NKPC (43)	`dsgeneural.py`	`_residuals_tf`, `R_pc1`
Taylor rule (44)	`dsgeneural.py`	`_residuals_tf`, `i_ = pp*pi + py_*y + v`
Char. denominator (68)	`dsgeEZ.py`	`_solve_first_order`, `denom(r)`
g_{ya} (69)	`dsgeEZ.py`	`_solve_first_order`, `g_ya`
AiO loss (77)	`dsgeneural.py`	`step_sob`, `aio`
Sobolev (79)	`dsgeneural.py`	`step_sob`, `sob`
PG (81)	`dsgeneural.py`	`step_sob`, `pg_loss`
GC (82)	`dsgeneural.py`	`step_sob`, `gc_val`
Reparameterisation (75)	`dsgeneural.py`	`_setup_posterior`, `_M_np`
Kalman predict (88)–(89)	`dsgeEZ.py`	`KalmanFilter.log_likelihood`
UKF sigma pts (96)	`dsgeEZ.py`	`SquareRootUKF`, `gen_sigma`
Pruned UKF (97)–(99)	`dsgeEZ.py`	`PrunedSquareRootUKF`
Param transforms	`dsgeEZ.py`	`transform_from_unconstrained`
Log-posterior (103)	`dsgeEZ.py`	`target_fn` in estimators
3-phase HMC	`dsgeEZ.py`	`run_hmc_chains`
Ridge Hessian	`dsgeneural.py`	`solve_regression`

20 Conclusion

This document has presented a complete framework for Bayesian estimation of DSGE models using neural network solvers, from household optimisation through to Hamiltonian Monte Carlo diagnostics.

Across 30 systematic experiments (22 in $\boldsymbol{\theta}$-space + 8 under canonical reparameterisation), we identify two complementary ingredients for a production-quality neural DSGE solver:

Canonical reparameterisation ($\psi = M^{-1}(\theta - \theta_{\text{mode}})$, $M = \Sigma^{1/2}$) is the single most impactful architectural decision. It decorrelates parameters and equalises gradient scales, reducing divergences 32-fold even without explicit gradient regularisation. Under reparameterisation, Sobolev regularisation alone passes all diagnostics (0 divergences, $\hat{R} = 1.035$), broadening the applicability of neural DSGE solvers to models without perturbation solutions.

Perturbation gradient regularisation (PG, $\lambda = 0.01$ under reparameterisation) provides a specific, smooth, low-variance target for the neural network's parameter derivatives. Combined with reparameterisation, it achieves the best overall results: 0 divergences, $\hat{R} = 1.018$, ESS = 283.

Several techniques from the PINN literature (DCGD, SIREN, spectral normalisation) remain ineffective regardless of parameter space, confirming that the gradient quality problem in DSGE solvers has a fundamentally different structure from PDE residual networks. The perturbation-corrected architecture is incompatible with reparameterisation and should not be combined.

The common thread linking all failures is a violation of a simple principle: *the neural network's derivatives with respect to structural parameters must be smooth and accurate, not just its function values.* The two most effective interventions—working in the right coordinate system (reparameterisation) and directly training the derivatives (PG)—both address this principle directly.

For scaling to large-scale models (ECB NAWM: $n_\theta \sim 80$), canonical reparameterisation is expected to become *more* valuable as the posterior covariance condition number grows. The automatic decorrelation provided by the eigendecomposition square root is precisely what high-dimensional samplers need: HMC trajectories in ψ-space explore decorrelated, unit-scale directions where the leapfrog integrator is maximally efficient.

A Detailed Documentation of `dsgeEZ.py`

This appendix documents the core framework file. The file is organised into major sections: backend abstraction (`BackendRegistry`, `TensorFlowBackend`), model classes (`SmallNKModel`, `EpsteinZinNKModel`, `SGUEstimable`), filter classes (`KalmanFilter`, `AugmentedKalmanFilter`, `SquareRootUKF`, `PrunedSquareRootUKF`), estimator classes (`MAPEstimator`, `HMCEstimator`, `MultiChainHMC`), diagnostic functions, and plotting utilities.

A.1 Backend and Utility Layer

`BackendRegistry` and `B()`. A registry pattern that stores a single global backend instance. `BackendRegistry.set_backend('tensorflow', precision='float64', force_cpu=True)` creates a `TensorFlowBackend`. The convenience function `B()` returns the active backend, used throughout the codebase as `b = B()`.

`TensorFlowBackend` class. Implements the `AbstractBackend` interface with TensorFlow and TensorFlow Probability. All constants and factory methods create tensors on `/CPU:0` explicitly to prevent XLA cross-device errors. Pre-allocates frequently-used scalar constants (`ZERO`, `ONE`, `HALF`, `LOG2PI`, `NUGGET`, `BIG_NEG`) at construction time. When `force_cpu=True` and `precision='float64'`, hides all GPUs via `tf.config.set_visible_devices([], 'GPU')`.

Pitfall: The float64 setting is determined by the backend, not by a global TF setting. All downstream code calls `b.tensor()`, `b.zeros()`, etc., which use the backend's dtype. This is cleaner than the original approach of setting `tf.keras.backend.set_floatx('float64')` globally.

The `compile(fn, jit=True)` method wraps `tf.function(fn, jit_compile=jit)`. When a GPU is visible (but force_cpu is False), it additionally wraps the compiled function to execute on CPU, preventing cross-device errors for MAP/HMC operations.

A.2 Model Classes

A.2.1 `SmallNKModel`

Inherits from `AbstractNonlinearDSGE`. The `nl_scale` attribute is set at construction time: `SmallNKModel(nl_scale=2.0)`. This controls the amplitude of second-order corrections in `_second_order`.

`solve(self, theta) -> ModelSolution`. Calls `_solve_first_order` and `_second_order`, returns a `ModelSolution` dataclass containing `g_x`, `h_x`, `eta`, `G_xx`, `H_xx`, `g_ss`, `h_ss`, and `R_obs`. The measurement noise $R_{\text{obs}} = 10^{-4} I_2$ is set inside `solve()`.

`build_augmented_ssm(self, sol) -> AugmentedStateSpace`. Constructs the augmented state-space form for the `AugmentedKalmanFilter`. The augmented state is $(x_t, x_t \otimes x_t, \tilde{x}_t)$ where $\tilde{x}_t$ accumulates second-order corrections.

`simulate(self, theta_np, T_obs=200, seed=42)`. Returns `(Y, X)` where `Y` is the $(T, 2)$ observation matrix and `X` is the $(T, 2)$ state matrix. Note: the simulation loop computes controls *before* updating the state, so `Y[t]` corresponds to `X[t]`, not `X[t+1]`.

A.2.2 `EpsteinZinNKModel`

A 10-parameter model with recursive Epstein–Zin preferences. Parameters: $(\beta, \gamma, \psi, \kappa, \phi_\pi, \phi_y, \rho_a, \rho_v, \sigma_a, \sigma_v)$. At first order, only ψ enters (via the IS curve, replacing $1/\sigma_c$ with ψ). All second-order corrections scale with $\delta = \gamma - 1/\psi$. Includes cross-state interaction terms and a risk premium

correction in g_{ss}. The simulation includes state clipping at $\pm 5\sigma$ to prevent divergence when δ is large.

A.3 Filter Classes

All filters implement `log_likelihood(self, model, data, theta)` returning a scalar log-likelihood. They accept the model and theta separately (rather than a pre-computed solution) and call `model.solve(theta)` internally.

The `KalmanFilter` uses the Joseph form for the covariance update and Cholesky-based log-determinant computation. The `SquareRootUKF` maintains the state covariance Cholesky factor directly, with QR-based prediction updates and general-dimension rank-1 Cholesky downdates for the measurement update. The `PrunedSquareRootUKF` implements a pruned UKF that tracks x^f as the UKF state and x^s as a deterministic side variable, following the decomposition of Andreasen et al. Andreasen et al. (2018); the UKF sigma-point propagation through the nonlinear observation (which includes the $\text{kron}(x^f, x^f)$ term) replaces the EKF linearisation used in earlier versions. This works well for models with moderate $\|H_{xx}\|$ (e.g., SmallNK). For the SGU model, where $\|H_{xx}\| \approx 604$ (see Section 6.4), first-order filtering is used instead (see Section 12.5 for discussion).

A.4 Estimator Classes

`MAPEstimator`. Uses Adam with gradient clipping (norm 100) for robustness. The loss and gradient are computed in a JIT-compiled function. Tracks the best log-posterior seen across all steps.

`HMCEstimator`. Single-chain HMC using TFP's `SimpleStepSizeAdaptation`. Used for quick preliminary runs (e.g., Stage 1 of the tutorial).

`MultiChainHMC`. Production-grade multi-chain HMC implementing the three-phase pipeline (Section 13.2.3). Returns a `MultiChainResult` dataclass containing per-chain samples, acceptance rates, step sizes, divergence counts, preconditioning scale, and timing information.

A.5 Diagnostic Functions

`compute_rhat(chains)`. Split-$\hat{R}$ with rank normalisation following Vehtari et al. (2021). Splits each chain in half, rank-normalises to normal scores, then computes the standard $\hat{R}$ formula.

`compute_ess(chains)`. Bulk ESS (from rank-normalised draws) and tail ESS (from folded rank-normalised draws). Uses the initial positive sequence estimator for autocorrelation.

`mcmc_summary(chains, param_names, theta_true)`. Prints a formatted table with posterior mean, std, quantiles, ESS, $\hat{R}$, and coverage for each parameter. Flags parameters with $\hat{R} > 1.01$ or ESS < 400.

B Detailed Documentation of `dsgeneural.py`

This appendix documents the neural solver file.

B.1 Network Architectures

B.1.1 `ScalablePolicyNetwork`

The production architecture. Auto-selects depth and width based on d_{in} (see Section 10.1). Supports custom activations (`tanh`, `mrepu2`, `mrepu3`, `siren`) via the `_resolve_activation` dispatcher, spectral normalisation via `spectral_norm=True`, and SIREN via delegation to `SIRENNetwork`. For $d_{\text{in}} \geq 60$, builds a `tf.keras.Model` with gated residual connections (skip every 2 layers when widths match).

B.1.2 `SIRENNetwork`

Multi-layer network with $\sin(\omega_0 z)$ activations and SIREN-specific initialisation. The first layer uses $W \sim U(-1/d_{\text{in}}, 1/d_{\text{in}})$; subsequent layers use $W \sim U(-\sqrt{6/d_\ell}/\omega_0, \sqrt{6/d_\ell}/\omega_0)$.

B.2 Solver Classes

B.2.1 `ConditionedNeuralNKSolver`

The main production class. Key attributes: model dimensions (`N_STATES=2`, `N_PARAMS=9`, etc.), structural bounds (`_BOUNDS_LO`, `_BOUNDS_HI`), and state-to-parameter index mappings (`_STATE_RHO_IDX`, `_STATE_SIG_IDX`).

`_setup_posterior`. When `reparam=True`: computes $M = V\Lambda^{1/2}$ from the eigendecomposition of the posterior covariance, stores M, M^{-1}, and θ_{mode} as both NumPy arrays and TF constants. Computes the training domain in ψ-space. When `reparam=False`: uses an inflated box around the posterior in θ-space.

`_sample_theta`. When `reparam=True`: samples ψ from the inflated posterior Gaussian via LHS, round-trips through θ for bound enforcement ($\psi \to \theta \to \text{clip} \to \psi$). When `reparam=False`: samples θ directly from the inflated posterior Gaussian.

`pretrain`. Warm-starts the NN with supervised targets. Uses dimension-dependent defaults from `pretrain_defaults`. When `reparam=True`, the pool of parameter vectors contains ψ values; these are converted to structural θ for the perturbation solver. Features: cosine LR decay, validation split with early stopping, curriculum learning (1st-order-only targets initially), periodic state-grid refresh, gradient clipping.

`train_sobolev`. Main AiO training loop with all regularisation options. The `step_sob` function uses nested `GradientTape`s: the outer tape computes $\partial\mathcal{L}/\partial\boldsymbol{\psi}$ for the optimiser; the inner persistent tape computes $\partial R/\partial\boldsymbol{\theta}$ for Sobolev, $\partial\Pi/\partial\boldsymbol{\theta}$ for PG, and $\partial R/\partial\boldsymbol{\theta}$ for GC. Supports DCGD mode (`dcgd=True`) via a separate `step_dcgd` function.

`solve(theta)`. External interface accepting structural θ. When `reparam=True`, converts $\theta \to \psi$ for NN evaluation and stores structural θ for the model solution. Dispatches to `solve_fd`, `solve_autodiff`, or `solve_regression`.

`solve_regression`. The default coefficient extraction method. Stage 1: FD Jacobian (central differences, $2n_s$ probes). Stage 2: Ridge-regression Hessian (random samples near origin, subtract known linear terms, regress quadratic). All NN evaluations are fused into a single batched forward pass. Fully JIT-safe.

migrate_to_cpu. Rebuilds the NN on /CPU:0 with weights copied from GPU. Also migrates the residual balancing log-variance variable and all reparameterisation tensors. Essential before HMC when training was done on GPU.

C Detailed Documentation of tutorial_neural_dsge.py

The tutorial runs three complete estimation pipelines sequentially.

C.1 Overall Structure

The tutorial defines a NeuralSmallNKModel wrapper class that inherits from SmallNKModel and delegates solve() to a trained ConditionedNeuralNKSolver. The run_map_hmc helper function runs MAP optimisation followed by multi-chain HMC for each case.

C.2 Case C: Neural Estimation (Three Stages)

Stage 1: Quick perturbation posterior. A short single-chain HMC (HMCEstimator, 200 draws + 100 burnin) using the perturbation model and UKF generates theta-samples for the neural solver's training domain.

Stage 2: Train reparameterised neural solver. Creates a ConditionedNeuralNKSolver with hidden=(128, 128) and the theta-samples from Stage 1. Pretrains from a SmallNKModel(nl_scale=NL_S perturbation model, then runs AiO + Sobolev + PG training (train_sobolev). The default reparam=True trains in ψ-space.

Stage 3: Full multi-chain HMC. Wraps the trained solver in NeuralSmallNKModel, pairs it with the UKF, and runs MultiChainHMC with the same settings as Cases A and B.

Pitfall: Case C uses seed=99 for data generation (different from Cases A and B which use seed=42) to avoid look-ahead bias: the neural solver is trained on theta-samples from the perturbation posterior, not from the same data it will be evaluated on.

References

Calvo, G. A. (1983). Staggered prices in a utility-maximizing framework. *Journal of Monetary Economics*, 12(3):383–398.

Chen, Z., Badrinarayanan, V., Lee, C.-Y., and Rabinovich, A. (2018). GradNorm: Gradient normalisation for adaptive loss balancing in deep multitask networks. In *ICML*, pp. 794–803.

Galí, J. (2015). *Monetary Policy, Inflation, and the Business Cycle*. Princeton University Press, 2nd edition.

Hoffman, M. D. and Gelman, A. (2014). The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *JMLR*, 15:1593–1623.

Hutchinson, M. F. (1990). A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics—Simulation and Computation*, 19(2):433–450.

Kendall, A., Gal, Y., and Cipolla, R. (2018). Multi-task learning using uncertainty to weigh losses. In *CVPR*, pp. 7482–7491.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *ICLR*.

Maliar, L., Maliar, S., and Winant, P. (2021). Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101.

Miyato, T., Kataoka, T., Koyama, M., and Yoshida, Y. (2018). Spectral normalisation for generative adversarial networks. In *ICLR*.

Neal, R. M. (2011). MCMC using Hamiltonian dynamics. In *Handbook of MCMC*, chapter 5. CRC.

Nesterov, Y. (2009). Primal-dual subgradient methods for convex problems. *Mathematical Programming*, 120(1):221–259.

Sitzmann, V., Martel, J. N. P., Bergman, A. W., Lindell, D. B., and Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. In *NeurIPS*, pp. 7462–7473.

Taylor, J. B. (1993). Discretion versus policy rules in practice. *Carnegie-Rochester Conference Series*, 39:195–214.

Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., and Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$. *Bayesian Analysis*, 16(2):667–718.

Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., and Finn, C. (2020). Gradient surgery for multi-task learning. In *NeurIPS*, pp. 5824–5836.

Andreasen, M. M., Fernández-Villaverde, J., and Rubio-Ramírez, J. F. (2018). The pruned state-space system for non-linear DSGE models: Theory and empirical applications. *Review of Economic Studies*, 85(1):1–49.

Kim, J., Kim, S., Schaumburg, E., and Sims, C. A. (2008). Calculating and using second-order accurate solutions of discrete time dynamic equilibrium models. *Journal of Economic Dynamics and Control*, 32(11):3397–3414.

Gill, P. E., Golub, G. H., Murray, W., and Saunders, M. A. (1974). Methods for modifying matrix factorizations. *Mathematics of Computation*, 28(126):505–535.

Van der Merwe, R. and Wan, E. A. (2001). The square-root unscented Kalman filter for state and parameter-estimation. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pp. 3461–3464.

Fernández-Villaverde, J. and Rubio-Ramírez, J. F. (2007). Estimating macroeconomic models: A likelihood approach. *Review of Economic Studies*, 74(4):1059–1087.

Schmitt-Grohé, S. and Uribe, M. (2003). Closing small open economy models. *Journal of International Economics*, 61(1):163–185.

Schmitt-Grohé, S. and Uribe, M. (2004). Solving dynamic general equilibrium models using a second-order approximation to the policy function. *Journal of Economic Dynamics and Control*, 28(4):755–775.

Childers, D., Fernández-Villaverde, J., Perla, J., Rackauckas, C., and Wu, P. (2024). Differentiable state-space models and Hamiltonian Monte Carlo estimation. Working paper, available at `https://github.com/HighDimensionalEconLab/HMCExamples.jl`.